本文档由 funstory.ai 的开源 PDF 翻译库 BabelDOC v0.6.2 (https://github.com/funstory-ai/BabelDOC) 翻译，本仓库正在积极的建设当中，欢迎 star 和关注。

arXiv:2205.14135v2 [cs.LG] 23 Jun 2022

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY
{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

June 24, 2022

Abstract

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this problem by trading off model quality to reduce the compute complexity, but often do not achieve wall-clock speedup. We argue that a missing principle is making attention algorithms *IO-aware*—accounting for reads and writes between levels of GPU memory. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. We analyze the IO complexity of FlashAttention, showing that it requires fewer HBM accesses than standard attention, and is optimal for a range of SRAM sizes. We also extend FlashAttention to block-sparse attention, yielding an approximate attention algorithm that is faster than any existing approximate attention method. FlashAttention trains Transformers faster than existing baselines: 15% end-to-end wall-clock speedup on BERT-large (seq. length 512) compared to the MLPerf 1.1 training speed record, 3× speedup on GPT-2 (seq. length 1K), and 2.4× speedup on long-range arena (seq. length 1K-4K). FlashAttention and block-sparse FlashAttention enable longer context in Transformers, yielding higher quality models (0.7 better perplexity on GPT-2 and 6.4 points of lift on long-document classification) and entirely new capabilities: the first Transformers to achieve better-than-chance performance on the Path-X challenge (seq. length 16K, 61.4% accuracy) and Path-256 (seq. length 64K, 63.1% accuracy).

1 Introduction

Transformer models [82] have emerged as the most widely used architecture in applications such as natural language processing and image classification. Transformers have grown larger [5] and deeper [83], but equipping them with longer context remains difficult [80], since the self-attention module at their heart has time and memory complexity quadratic in sequence length. An important question is whether making attention faster and more memory-efficient can help Transformer models address their runtime and memory challenges for long sequences.

Many approximate attention methods have aimed to reduce the compute and memory requirements of attention. These methods range from sparse-approximation [51, 74] to low-rank approximation [12, 50, 84], and their combinations [3, 9, 92]. Although these methods reduce the compute requirements to linear or near-linear in sequence length, many of them do not display wall-clock speedup against standard attention and have not gained wide adoption. One main reason is that they focus on FLOP reduction (which may not correlate with wall-clock speed) and tend to ignore overheads from memory access (IO).

In this paper, we argue that a missing principle is making attention algorithms *IO-aware* [1]—that is, carefully accounting for reads and writes to different levels of fast and slow memory (e.g., between fast GPU on-chip SRAM and relatively slow GPU high bandwidth memory, or HBM [45], Figure 1 left). On modern

arXiv:2205.14135v2 [cs.LG] 23 Jun 2022

FlashAttention：快速且内存高效的精确注意力机制与IO感知

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], 和 Christopher Ré [†]

[†]斯坦福大学计算机科学系
[‡]纽约州立大学布法罗分校计算机科学与工程学院 {trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

2022年6月24日

摘要

Transformer在长序列上运行缓慢且内存消耗大，因为自注意力的时间和内存复杂度与序列长度呈平方关系。近似注意力方法试图通过牺牲模型质量来降低计算复杂度以解决这个问题，但通常无法实现时钟速度提升。我们认为一个缺失的原则是使注意力算法<样式 id='1'>IO感知</样式>——考虑GPU内存各级之间的读写操作。我们提出了FlashAttention，这是一种IO感知的精确注意力算法，它使用tiling技术来减少GPU高带宽内存（HBM）和GPU片上SRAM之间的内存读写次数。我们分析了FlashAttention的IO复杂度，表明它比标准注意力需要的HBM访问次数更少，并且对于一系列SRAM大小都是最优的。我们还扩展了FlashAttention到块稀疏注意力，产生了一种比任何现有近似注意力方法都快的大致注意力算法。FlashAttention比现有基线训练Transformer更快：在BERT-large（序列长度512）上比MLPerf 1.1训练速度记录快15%的端到端时钟速度提升，在GPT-2（序列长度1K）上快 3×倍，在长程竞技场（序列长度1K-4K）上快2.4× 倍。FlashAttention和块稀疏FlashAttention使Transformer的上下文更长，从而产生更高质量的模型（在GPT-2上困惑度降低0.7，在长文档分类上提升6.4分）并实现全新的能力：首个在Path-X挑战（序列长度16K，61.4%准确率）和Path-256（序列长度64K，63.1%准确率）上表现优于随机猜测的Transformer。

1 引言

Transformer模型 [82] 已成为自然语言处理和图像分类等应用中最广泛使用的架构。Transformer模型变得更大 [5] 且更深 [83]，，但为它们配备更长的上下文仍然很困难 [80]，，因为它们核心的自注意力模块的时间复杂度和内存复杂度都是序列长度的二次方。一个重要的问题是，是否可以通过使注意力更快、更内存高效来帮助Transformer模型解决长序列的运行时和内存挑战。

许多近似注意力方法旨在减少注意力的计算和内存需求。这些方法范围从稀疏近似 [51, 74] 到低秩近似 [12, 50, 84],及其组合 [3, 9, 92]。虽然这些方法将计算需求降低到序列长度的线性或近线性，但其中许多方法在标准注意力上没有显示时钟速度提升，并且没有获得广泛采用。一个主要原因是它们专注于FLOP减少（这可能不与时钟速度相关），并且倾向于忽略内存访问（IO）的开销。

在本文中，我们主张一个缺失的原则正在使注意力算法 *IO感知* [1]——也就是说，仔细考虑对不同级别快速和慢速内存的读写（例如，在快速GPU片上SRAM和相对较慢的GPU高带宽内存HBM [45], 图1左侧之间）。在现代

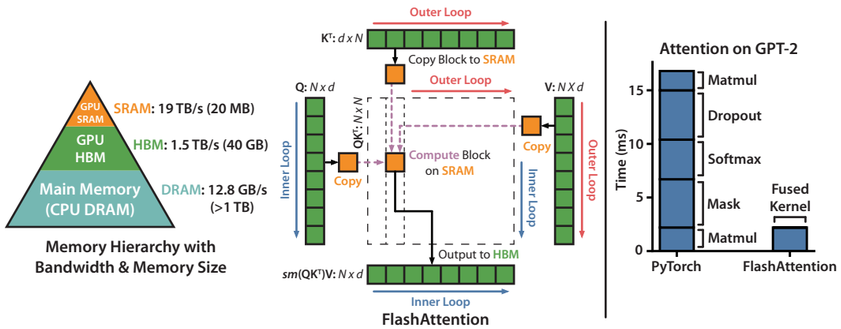


Figure 1: **Left:** FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the **K** and **V** matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

GPUs, compute speed has out-paced memory speed [61, 62, 63], and most operations in Transformers are bottlenecked by memory accesses [43]. IO-aware algorithms have been critical for similar memory-bound operations, when reading and writing data can account for a large portion of the runtime—such as database joins [71], image processing [70], numerical linear algebra [4], and more [40, 85]. However, common Python interfaces to deep learning such as PyTorch and Tensorflow do not allow fine-grained control of memory access.

We propose FlashAttention, a new attention algorithm that computes exact attention with far fewer memory accesses. Our main goal is to avoid reading and writing the attention matrix to and from HBM. This requires (i) computing the softmax reduction without access to the whole input (ii) not storing the large intermediate attention matrix for the backward pass. We apply two well-established techniques to address these challenges. (i) We restructure the attention computation to split the input into blocks and make several passes over input blocks, thus incrementally performing the softmax reduction (also known as **tiling**). (ii) We store the softmax normalization factor from the forward pass to quickly **recompute** attention on-chip in the backward pass, which is faster than the standard approach of reading the intermediate attention matrix from HBM. We implement FlashAttention in CUDA to achieve fine-grained control over memory access and fuse all the attention operations into one GPU kernel. Even with the increased FLOPs due to recomputation, our algorithm both **runs faster** (up to 7.6x on GPT-2 [67], Figure 1 right) and **uses less memory**—linear in sequence length—than standard attention, thanks to the massively reduced amount of HBM access.

We analyze the IO complexity [1] of FlashAttention, proving that it requires $O(N^2 d^2 M^{-1})$ HBM accesses where d is the head dimension and M is the size of SRAM, as compared to $\Omega(Nd + N^2)$ of standard attention. For typical values of d and M, FlashAttention requires many times fewer HBM accesses compared to standard attention (up to 9× fewer, as shown in Fig. 2). Moreover, we provide a lower bound, showing that no exact attention algorithm can asymptotically improve on the number of HBM accesses over all SRAM sizes.

We also show that FlashAttention can serve as a useful primitive for realizing the potential of approximate attention algorithms by overcoming their issues with memory access overhead. As a proof of concept, we implement block-sparse FlashAttention, a sparse attention algorithm that is 2-4× faster than even FlashAttention, scaling up to sequence length of 64k. We prove that block-sparse FlashAttention has better IO complexity than FlashAttention by a factor proportional to the sparsity ratio. We discuss further extensions to other operations (attention on multi-GPU, kernel regression, block-sparse matrix

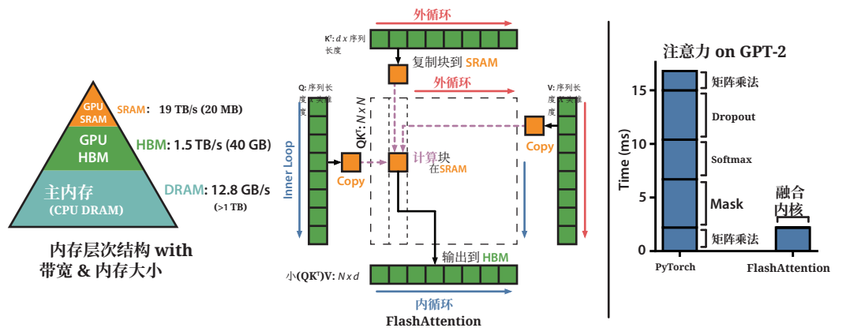


图1： **左：** FlashAttention使用tiling来防止在（相对）较慢的GPU HBM上实现大型 $N \times N$ 注意力矩阵（虚线框）。在外循环（红色箭头）中，FlashAttention遍历**K** 和 **V** 矩阵的块，并将它们加载到快速的片上SRAM。在每个块中，FlashAttention遍历 **Q** 矩阵的块（蓝色箭头），将它们加载到SRAM，并将注意力计算的输出写回HBM。 **右：** 相对于GPT-2上PyTorch注意力实现的加速比。FlashAttention不会读取和写入大型 $N \times N$ 注意力矩阵到HBM，从而在注意力计算上实现了7.6×的加速比。

GPU，计算速度已经超越了内存速度 [61, 62, 63], ，并且 Transformer 中的大多数操作都受到内存访问 [43] 的瓶颈。对于类似的内存绑定操作，IO 感知的算法至关重要，当读写数据可以占运行时的一大部分时——例如数据库连接 [71]，图像处理 [70]，数值线性代数 [4]，以及更多 [40, 85]。然而，深度学习的常见 Python 接口如 PyTorch 和 Tensorflow 并不允许对内存访问进行细粒度控制。

我们提出了FlashAttention，一种新的注意力算法，它通过远更少的内存访问来计算精确注意力。我们的主要目标是避免将注意力矩阵读入和写出到HBM。这需要 (i) 在无法访问整个输入的情况下计算softmax归约 (ii) 不为反向传递存储大型中间注意力矩阵。我们应用了两种成熟的技巧来解决这些挑战。(i) 我们重构了注意力计算，将输入分成块并在输入块上执行多次传递，从而逐步执行softmax归约（也称为<样式 id='1'>tiling</样式>）。(ii) 我们存储了正向传递的softmax归一化因子，以便在反向传递中快速在片上重新计算注意力，这比从HBM读取中间注意力矩阵的标准方法更快。我们在CUDA中实现了FlashAttention，以实现对内存访问的细粒度控制，并将所有注意力操作融合到一个GPU内核中。即使由于重新计算导致FLOPs增加，我们的算法也同时<样式 id='5'>运行更快</样式>（在GPT-2 [67], Figure 1右侧最高可达7.6倍）和<样式 id='8'>使用更少内存</样式>——与序列长度呈线性关系——这得益于HBM访问量的大幅减少，从而优于标准注意力。

我们分析了FlashAttention的I/O复杂度 [1] ，证明其需要 $O(N^2 d^2 M^{-1})$次HBM访问，其中 d 是头维度，M 是SRAM的大小，相比之下，标准注意力需要 $\Omega(Nd + N^2)$次。对于 d 和 M的典型值，FlashAttention与标准注意力相比，所需的HBM访问次数少很多（最多少 9× 次，如图2所示）。此外，我们提供了一个下界，表明对于所有SRAM大小，没有精确注意力算法可以在渐近意义上改进HBM访问次数。

我们还证明，FlashAttention可以通过克服近似注意力算法在内存访问开销方面的问题，作为实现近似注意力算法潜力的有用基础。作为一个概念验证，我们实现了块稀疏FlashAttention，这是一种比FlashAttention快2-4× 倍的稀疏注意力算法，可扩展到64k的序列长度。我们证明，块稀疏FlashAttention的I/O复杂度比FlashAttention好，其改进程度与稀疏性比率成正比。我们讨论了进一步扩展到其他操作（多GPU上的注意力、内核回归、块稀疏矩阵

multiply) in Section 5. We open-source FlashAttention to make it easier to build on this primitive.[1]

We empirically validate that FlashAttention speeds up model training and improves model quality by modeling longer context. We also benchmark the runtime and memory footprint of FlashAttention and block-sparse FlashAttention compared to prior attention implementations.

- **Faster Model Training.** FlashAttention trains Transformer models faster in wall-clock time. We train BERT-large (seq. length 512) 15% faster than the training speed record in MLPerf 1.1 [58], GPT2 (seq. length 1K) 3× faster than baseline implementations from HuggingFace [87] and Megatron-LM [77], and long-range arena (seq. length 1K-4K) 2.4× faster than baselines.
- **Higher Quality Models.** FlashAttention scales Transformers to longer sequences, which improves their quality and enables new capabilities. We observe a 0.7 improvement in perplexity on GPT-2 and 6.4 points of lift from modeling longer sequences on long-document classification [13]. FlashAttention enables the first Transformer that can achieve better-than-chance performance on the Path-X [80] challenge, solely from using a longer sequence length (16K). Block-sparse FlashAttention enables a Transformer to scale to even longer sequences (64K), resulting in the first model that can achieve better-than-chance performance on Path-256.
- **Benchmarking Attention.** FlashAttention is up to 3× faster than the standard attention implementation across common sequence lengths from 128 to 2K and scales up to 64K. Up to sequence length of 512, FlashAttention is both faster and more memory-efficient than any existing attention method, whereas for sequence length beyond 1K, some approximate attention methods (e.g., Linformer) start to become faster. On the other hand, block-sparse FlashAttention is faster than all existing approximate attention methods that we know of.

2 Background

We provide some background on the performance characteristics of common deep learning operations on modern hardware (GPUs). We also describe the standard implementation of attention.

2.1 Hardware Performance

We focus here on GPUs. Performance on other hardware accelerators are similar [46, 48].

GPU Memory Hierarchy. The GPU memory hierarchy (Fig. 1 left) comprises multiple forms of memory of different sizes and speeds, with smaller memory being faster. As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [44, 45]. The on-chip SRAM is an order of magnitude faster than HBM but many orders of magnitude smaller in size. As compute has gotten faster relative to memory speed [61, 62, 63], operations are increasingly bottlenecked by memory (HBM) accesses. Thus exploiting fast SRAM becomes more important.

Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

Performance characteristics. Depending on the balance of computation and memory accesses, operations can be classified as either compute-bound or memory-bound. This is commonly measured by the *arithmetic intensity* [85], which is the number of arithmetic operations per byte of memory access.

1. Compute-bound: the time taken by the operation is determined by how many arithmetic operations there are, while time accessing HBM is much smaller. Typical examples are matrix multiply with large inner dimension, and convolution with large number of channels.
2. Memory-bound: the time taken by the operation is determined by the number of memory accesses, while time spent in computation is much smaller. Examples include most other operations: elementwise (e.g., activation, dropout), and reduction (e.g., sum, softmax, batch norm, layer norm).

Kernel fusion. The most common approach to accelerate memory-bound operations is kernel fusion: if there are multiple operations applied to the same input, the input can be loaded once from HBM, instead of multiple times for each operation. Compilers can automatically fuse many elementwise operations [53, 65, 75].

[1] FlashAttention code is available at `https://github.com/HazyResearch/flash-attention`

乘法) 在第 在5上。我们开源FlashAttention，使其更容易基于这个基础进行构建。[1]

我们通过实证验证，FlashAttention 通过建模更长的上下文来加速模型训练并提高模型质量。我们还对比了 FlashAttention 和块稀疏 FlashAttention 与先前注意力实现的运行时和内存占用。

- **更快的模型训练**。 FlashAttention 在墙上时间（wall-clock time）中更快地训练 Transformer 模型。我们训练 BERT-large（序列长度 512）比 MLPerf 1.1 [58],GPT2（序列长度 1K） 3× 比 HuggingFace [87] 和 Megatron-LM [77],的基线实现快 15%，比长程竞技场（序列长度 1K-4K）的基线实现快 2.4× 。
- **更高质量的模型**。 FlashAttention 将 Transformer 扩展到更长的序列，这提高了它们的质量并实现了新功能。我们在 GPT-2 上观察到困惑度（perplexity）提高了 0.7，在长文档分类任务上通过建模更长的序列提升了 6.4 个点 [13]。FlashAttention 使第一个 Transformer 能够在 Path-X [80] 挑战中取得比随机猜测更好的表现，仅仅通过使用更长的序列长度（16K）。块稀疏 FlashAttention 使 Transformer 能够扩展到更长的序列（64K），从而实现了第一个在 Path-256 上取得比随机猜测更好表现的模型。
- **基准测试注意力**。 FlashAttention 比标准注意力实现快高达 3× 倍，在 128 到 2K 的常见序列长度范围内，并且扩展到 64K。在序列长度高达 512 时，FlashAttention 既是更快的，也是比任何现有注意力方法更内存高效的，而对于序列长度超过 1K 的，一些近似注意力方法（例如 Linformer）开始变得更快。另一方面，块稀疏 FlashAttention 比我们已知的所有现有近似注意力方法都快。

2 背景

我们提供了一些关于现代硬件（GPU）上常见深度学习操作的性能特征背景。我们还描述了注意力的标准实现。

2.1 硬件性能

We 这里关注 GPU。其他硬件加速器的性能类似 [46, 48]。

GPU 内存层次结构。 GPU 内存层次结构（图 1 左侧）包含多种不同大小和速度的内存形式，其中较小的内存更快。例如，A100 GPU 拥有 40-80GB 的高带宽内存（HBM），带宽为 1.5-2.0TB/s，并且每个 108 个流式多处理器都有 192KB 的片上 SRAM，带宽估计约为 19TB/s [44, 45]。片上 SRAM 比内存（HBM）快一个数量级，但在大小上小很多个数量级。随着计算速度相对于内存速度 [61, 62, 63]，变得更快，操作越来越多地受到内存（HBM）访问的瓶颈。因此，利用快速 SRAM 变得更加重要。

执行模型。 GPU拥有大量线程来执行一个操作（称为内核）。每个内核从HBM加载输入到寄存器和SRAM，进行计算，然后将输出写入HBM。

性能特征。 根据计算和内存访问的平衡程度，操作可以被分类为计算密集型或内存密集型。这通常通过算术强度 [85], 来衡量，即每字节数据访问的算术运算次数。

1. 计算密集型：操作的耗时由算术运算的数量决定，而访问HBM的时间则小得多。典型例子包括内维度较大的矩阵乘法，以及通道数量较多的卷积。

2. 内存密集型：操作的耗时由内存访问的数量决定，而计算所花费的时间则小得多。例子包括大多数其他操作：逐元素操作（例如，激活、Dropout），以及归约操作（例如，求和、Softmax、批归一化、层归一化）。

内核融合。 加速内存绑定操作的最常见方法是内核融合：如果多个操作应用于同一输入，输入可以从 HBM 一次性加载，而不是为每个操作多次加载。编译器可以自动融合许多逐元素操作 [53, 65, 75]。

[1] FlashAttention 代码可在 https://github.com/HazyResearch/flash-attention 获取

However, in the context of model training, the intermediate values still need to be written to HBM to save for the backward pass, reducing the effectiveness of naive kernel fusion.

2.2 Standard Attention Implementation

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (e.g., for GPT2, $N = 1024$ and $d = 64$). We describe the standard attention implementation in Algorithm 0. As some or most of the operations are memory-bound (e.g., softmax), the large number of memory accesses translates to slow wall-clock time.

This problem is exacerbated by other elementwise operations applied to the attention matrix, such as masking applied to $\mathbf{S}$ or dropout applied to $\mathbf{P}$. As a result, there have been many attempts to fuse several elementwise operations, such as fusing masking with softmax [77].

In Section 3.2, we will show that the standard attention implementation performs HBM accesses quadratic in the sequence length N. We also compare the number of FLOPs and number of HBM accesses of standard attention and of our method (FlashAttention).

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.
1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

3 FlashAttention: Algorithm, Analysis, and Extensions

We show how to compute exact attention with fewer HBM reads/writes and without storing large intermediate matrices for the backward pass. This yields an attention algorithm that is both memory efficient and faster in wall-clock time. We analyze its IO complexity, showing that our method requires much fewer HBM accesses compared to standard attention. We further show that FlashAttention can serve as a useful primitive by extending it to handle block-sparse attention.

We focus here on the forward pass for ease of exposition; Appendix B contains details for the backward.

3.1 An Efficient Attention Algorithm With Tiling and Recomputation

Given the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, we aim to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$ and write it to HBM. Our goal is to reduce the amount of HBM accesses (to sub-quadratic in N).

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea is that we split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks, load them from slow HBM to fast SRAM, then compute the attention output with respect to those blocks. By scaling the output of each block by the right normalization factor before adding them up, we get the correct result at the end.

Tiling. We compute attention by blocks. Softmax couples columns of $\mathbf{K}$, so we decompose the large softmax with scaling [51, 60, 66]. For numerical stability, the softmax of vector $x \in \mathbb{R}^B$ is computed as:

$$m(x) := \max_i \; x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix}, \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

然而，在模型训练的上下文中，中间值仍需写入 HBM 以保存用于反向传播，这降低了朴素内核融合的有效性。

2.2 标准注意力实现

给定输入序列 $\mathbf{Q}$， $\mathbf{K}$， $\mathbf{V} \in \mathbb{R}^{N \times d}$ 其中 N 是序列长度， d 是头维度，我们希望计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

其中按行应用 softmax 函数。

标准注意力实现将矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 加载到 HBM 中，这需要 $O(N^2)$ 内存。通常 $N \gg d$ （例如，对于 GPT2, $N = 1024$ 和 $d = 64$）。我们描述了算法 0 中的标准注意力实现。由于部分或大部分操作受内存限制（例如，softmax），大量的内存访问导致实际运行时间变慢。

这个问题被应用于注意力矩阵的其他逐元素操作所加剧，例如对 $\mathbf{S}$ 应用掩码或对$\mathbf{P}$应用 dropout。因此，人们已经尝试融合多个逐元素操作，例如将掩码与 softmax [77] 融合。

在 3.2 节中，我们将证明标准注意力实现的 HBM 访问量与序列长度 N 的平方成正比。我们还比较了标准注意力和我们的方法（FlashAttention）的 FLOPs 数量和 HBM 访问数量。

算法0 标准注意力实现

要求： 矩阵 $\mathbf{Q}$， $\mathbf{K}$， $\mathbf{V} \in \mathbb{R}^{N \times d}$ 在HBM中。
1: 从HBM按块加载 $\mathbf{Q}$，$\mathbf{K}$，计算 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$，将 $\mathbf{S}$ 写入HBM。2: 从HBM读取 $\mathbf{S}$，计算 $\mathbf{P}$ =softmax($\mathbf{S}$)，将 $\mathbf{P}$ 写入HBM。3: 从HBM按块加载 $\mathbf{P}$ 和 $\mathbf{V}$，计算 $\mathbf{O}$ =$\mathbf{P}\mathbf{V}$，将 $\mathbf{O}$ 写入HBM。4: 返回$\mathbf{O}$。

3 FlashAttention：算法、分析与扩展

我们展示了如何用更少的HBM读写次数来计算精确注意力，并且无需为反向传递存储大型中间矩阵。这产生了一种内存高效且在墙上时间更快（wall-clock time）的注意力算法。我们分析了它的I/O复杂度，表明我们的方法与标准注意力相比需要大大减少HBM访问次数。我们进一步展示了FlashAttention可以作为一个有用的原语，通过将其扩展到处理块稀疏注意力。

我们在此关注　　为便于说明，采用前向传递；附录B包含反向传递的详细信息　　.

3.1 A一种带有tiling和recomputation的高效注意力算法　　n

给定HBM中的输入<样式id='1'>Q</样式><样式id='3'>、</样式><样式id='5'>K</样式><样式id='7'>、</样式><样式id='9'>V</样式> $\in \mathbb{R}^{N \times d}$，我们的目标计算注意力输出<样式id='12'>O</样式> $\in \mathbb{R}^{N \times d}$并将其写入HBM。我们的目标是将HBM访问次数减少到 N次以下二次方。

我们应用两种成熟的技术（tiling、recomputation）来克服在低于二次方HBM访问次数中计算精确注意力的技术挑战。我们将其描述在算法1中。主要思想是：我们将输入<样式id='1'>Q</样式><样式id='3'>、</样式><样式id='5'>K</样式><样式id='7'>、</样式><样式id='9'>V</样式>分成块，从慢速HBM加载到快速SRAM，然后计算这些块的注意力输出。通过在将它们相加之前，将每个块的输出乘以正确的归一化因子，我们最终得到正确的结果。

tiling。 我们通过分块计算注意力。Softmax将K的列耦合，因此我们使用缩放 [51, 60, 66]分解大型softmax。为了数值稳定性，向量的softmax $x \in \mathbb{R}^B$ 计算为:

$$m(x) := \max_i \; x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix}, \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)}\ x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)}\ell(x^{(1)}) + e^{m(x^{(2)})-m(x)}\ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

Therefore if we keep track of some extra statistics $(m(x), \ell(x))$, we can compute softmax one block at a time.[2] We thus split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks (Algorithm 1 line 3), compute the softmax values along with extra statistics (Algorithm 1 line 10), and combine the results (Algorithm 1 line 12).

Recomputation. One of our goals is to not store $O(N^2)$ intermediate values for the backward pass. The backward pass typically requires the matrices $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ to compute the gradients with respect to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$. However, by storing the output $\mathbf{O}$ and the softmax normalization statistics (m, ℓ), we can recompute the attention matrix $\mathbf{S}$ and $\mathbf{P}$ easily in the backward pass from blocks of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ in SRAM. This can be seen as a form of selective gradient checkpointing [10, 34]. While gradient checkpointing has been suggested to reduce the maximum amount of memory required [66], all implementations (that we know off) have to trade speed for memory. In contrast, even with more FLOPs, our recomputation speeds up the backward pass due to reduced HBM accesses (Fig. 2). The full backward pass description is in Appendix B.

Implementation details: Kernel fusion. Tiling enables us to implement our algorithm in one CUDA kernel, loading input from HBM, performing all the computation steps (matrix multiply, softmax, optionally masking and dropout, matrix multiply), then write the result back to HBM (masking and dropout in Appendix B). This avoids repeatedly reading and writing of inputs and outputs from and to HBM.

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

We show FlashAttention's correctness, runtime, and memory requirement (proof in Appendix C).

Theorem 1. *Algorithm 1 returns* $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ *with* $O(N^2 d)$ *FLOPs and requires* $O(N)$ *additional memory beyond inputs and output.*

3.2 Analysis: IO Complexity of FlashAttention

We analyze the IO complexity of FlashAttention, showing significant reduction in HBM accesses compared to standard attention. We also provide a lower bound, proving that no exact attention algorithm can

[2] This style of aggregation is called *algebraic aggregation* [33].

对于向量 $x^{(1)}$, $x^{(2)} \in \mathbb{R}^B$, 我们可以将拼接的 $x = \left[x^{(1)} x^{(2)}\right] \in \mathbb{R}^{2B}$的softmax分解为:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)}\ell(x^{(1)}) + e^{m(x^{(2)})-m(x)}\ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

因此如果我们跟踪一些额外的统计信息（$m(x)$, $l(x)$），我们可以一次计算一个块的softmax。[2]因此我们将输入 $\mathbf{Q}$,$\mathbf{K}$,$\mathbf{V}$ 分成块（算法1第3行），计算softmax值以及额外的统计信息（算法1第10行），然后组合结果（算法1第12行）。

重新计算。我们的一个目标是不要为反向传递存储 $O(N^2)$ 中间值。反向传递通常需要矩阵 $\mathbf{S}$, $\mathbf{P}$ $\in \mathbb{R}^{N \times N}$来计算相对于 $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ 的梯度。然而，通过存储输出 $\mathbf{O}$ 和softmax归一化统计（m, l），我们可以在反向传递中轻松地从SRAM中的 $\mathbf{Q}$,$\mathbf{K}$,$\mathbf{V}$ 块中重新计算注意力矩阵 $\mathbf{S}$ 和 $\mathbf{P}$ 。这可以看作是一种选择性梯度检查点 [10, 34]。虽然梯度检查点已被建议以减少所有实现（据我们所知）所需的内存最大量 [66]„ 但所有实现都必须在速度和内存之间进行权衡。相比之下，即使FLOPs更多，我们的重新计算也由于减少了HBM访问而加速了反向传递（图2）。完整的反向传递描述在附录B中。

实现细节：内核融合。分块使我们能够在单个CUDA内核中实现我们的算法，从HBM加载数据，执行所有计算步骤（矩阵乘法、Softmax、可选的掩码和Dropout、矩阵乘法），然后将结果写回HBM（掩码和Dropout在附录B中）。这避免了反复从HBM读取和写入输入和输出。

算法1 FlashAttention

要求： 矩阵 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$ $\in \mathbb{R}^{N \times d}$在HBM和片上SRAM M中。
1: 设置块大小 $B_c = \lceil \frac{M}{4d} \rceil, B_r =$的最小值 $\left(\lceil \frac{M}{4d} \rceil, d\right)$。 2: 初始化O $= (0)_{N \times d} \in \mathbb{R}^{N \times d}, \mathrm{l} = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$在HBM中。 3: 将Q分成 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$个大小为 $B_r \times d$的块Q_1,。 . . , Q_{T_r}, 将K、V分成 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$个大小为 $B_c \times d$的块K_1,。 . . , K_{T_c}和V_1,。 . . , V_{T_c}。 4: 将O分成 T_r个大小为 $B_r \times d$的块$\mathrm{O}_i, \ldots$, O_{T_r}, 将l分成 T_r 个大小为 B_r 的块 $\mathrm{l}_i, \ldots, \mathrm{l}_{T_r}$, 将 m 分成 T_r 个大小为 B_r 的块 $m_1, \ldots, m_{T_r}$。 5: f or $1 \le j \le T_c$ do 6: 将K $_j$, V $_j$从HBM加载到片上SRAM。 7: for $1 \le i \le T_r$ do 8: 将Q_i, O_i, l_i, m_i从HBM加载到片上SRAM。 9: 在芯片上计算$\mathrm{S}_{ij} = \mathrm{Q}_i \mathrm{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。 10: 在芯片上计算 $\tilde{m}_{ij} =$rowmax(S_{ij}) $\in \mathbb{R}^{B_r}$, $\tilde{\mathrm{P}}_{ij} =$exp($\mathrm{S}_{ij} - \tilde{m}_{ij}$) $\in \mathbb{R}^{B_r \times B_c}$(逐元素), $\tilde{\mathrm{l}}_{ij} =$rowsum($\tilde{\mathrm{P}}_{ij}$) $\in \mathbb{R}^{B_r}$。 11: 在芯片上计算 $m_i^{\text{new}} =$ $\max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \mathrm{l}_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \mathrm{l}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathrm{l}}_{ij} \in \mathbb{R}^{B_r}$。 12: 将$\mathrm{O}_i \leftarrow \text{diag}(\mathrm{l}_i^{\text{new}})^{-1}(\text{diag}(\mathrm{l}_i) e^{m_i - m_i^{\text{new}}} \mathrm{O}$ $_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathrm{P}}_{ij} \mathrm{V}_j$)写入HBM。 13: 将$\mathrm{l}_i \leftarrow \mathrm{l}_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$写入HBM。 14: end for 15: end for 16: 返回 O。

我们展示了 FlashAttention FlashAttention的正确性、运行时和内存需求（证明在附录C中） .

定理1。 算法*1*返回$\mathbf{O}$ $= \text{Softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$，其计算复杂度为 $O(N^2 d)$ *FLOPs*，并需要额外内存，超出输入和输出所需。

3.2 分析：FlashAttention的I/O复杂度

我们分析了FlashAttention的I/O复杂度，表明与标准注意力相比，其HBM访问显著减少。我们还提供了一个下界，证明没有任何精确注意力算法可以在所有SRAM大小上对HBM访问进行渐进式改进。

[2] 这种聚合方式称为代数聚合 [33]。

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

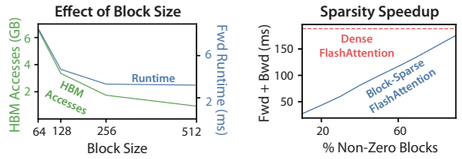


Figure 2: **Left**: Forward + backward runtime of standard attention and FlashAttention for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle**: Forward runtime of FlashAttention (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right**: The runtime (for seq. length 4K) of block-sparse FlashAttention is faster than FlashAttention by a factor proportional to the sparsity.

asymptotically improve on HBM accesses over all SRAM sizes. Proofs are in Appendix C.

Theorem 2. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *(Algorithm 1) requires $\Theta(N^2d^2M^{-1})$ HBM accesses.*

For typical values of d (64-128) and M (around 100KB), d^2 is many times smaller than M, and thus FlashAttention requires many times fewer HBM accesses than standard implementation. This leads to both faster execution and lower memory footprint, which we validate in Section 4.3.

The main idea of the proof is that given the SRAM size of M, we can load blocks of $\mathbf{K}, \mathbf{V}$ of size $\Theta(M)$ each (Algorithm 1 line 6). For each block of $\mathbf{K}$ and $\mathbf{V}$, we iterate over all blocks of $\mathbf{Q}$ (Algorithm 1 line 8) to compute the intermediate values, resulting in $\Theta(NdM^{-1})$ passes over $\mathbf{Q}$. Each pass loads $\Theta(Nd)$ elements, which amounts to $\Theta(N^2d^2M^{-1})$ HBM accesses. We similarly prove that the backward pass of standard attention requires $\Theta(Nd + N^2)$ HBM accesses while the backward pass of FlashAttention requires $\Theta(N^2d^2M^{-1})$ HBM accesses (Appendix B).

We prove a lower-bound: one cannot asymptotically improve on the number of HBM accesses for all values of M (the SRAM size) when computing exact attention.

Proposition 3. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. There does not exist an algorithm to compute exact attention with $o(N^2d^2M^{-1})$ HBM accesses for all M in the range $[d, Nd]$.*

The proof relies on the fact that for $M = \Theta(Nd)$ any algorithm must perform $\Omega(N^2d^2M^{-1}) = \Omega(Nd)$ HBM accesses. This type of lower bound over a subrange of M is common in the streaming algorithms literature [88]. We leave proving parameterized complexity [27] lower bounds in terms of M as exciting future work.

We validate that the number of HBM accesses is the main determining factor of attention run-time. In Fig. 2 (left), we see that even though FlashAttention has higher FLOP count compared to standard attention (due to recomputation in the backward pass), it has much fewer HBM accesses, resulting in much faster runtime. In Fig. 2 (middle), we vary the block size B_c of FlashAttention, which results in different amounts of HBM accesses, and measure the runtime of the forward pass. As block size increases, the number of HBM accesses decreases (as we make fewer passes over the input), and runtime decreases. For large enough block size (beyond 256), the runtime is then bottlenecked by other factors (e.g., arithmetic operations). Moreover, larger block size will not fit into the small SRAM size.

3.3 Extension: Block-Sparse FlashAttention

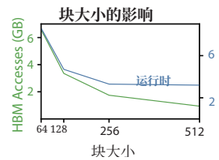
We extend FlashAttention to approximate attention: we propose block-sparse FlashAttention, whose IO complexity is smaller than FlashAttention by a factor proportional to the sparsity.

Given inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ and a mask matrix $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$, we want to compute:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\tilde{\mathbf{M}}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some block sizes B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor, j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

注意力	标准	FlashAttention
GFLOPs	66.6	75.2
HBM读写（GB）	40.3	4.4
运行时（毫秒）	41.7	7.3



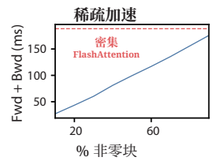


图2：**左**：标准注意力和FlashAttention在GPT-2中型（序列长度1024，头维度64，16个头，批量大小64）上的正向 + 反向运行时。HBM访问是影响运行时的主要因素。**中**：FlashAttention的正向运行时（序列长度1024，头维度64，16个头，批量大小64）在A100 GPU上。HBM访问次数越少，运行时越快，直到某个点。**右**：块稀疏FlashAttention的运行时（序列长度4K）比FlashAttention快，快多少与稀疏性成正比。

asymptotically improve on HBM访问 over all SRAM sizes. Proofs are in 附录C .

定理2。设 N 为序列长度， d 为头维度， M 为*SRAM*大小（含 $d \le M \le Nd$）。标准注意力（算法*0*）需要 $\Theta(Nd + N^2)$ *HBM*访问，而FlashAttention （算法*1*）需要 $\Theta(N^2d^2M^{-1})$ *HBM*访问。

对于典型的 d (64-128)和 M (约100KB)值， d^2 比 M小很多倍，因此FlashAttention比标准实现需要的HBM访问次数少很多。这导致了更快的执行速度和更低的内存占用，我们在第4.3节中对此进行了验证。

该证明的主要思想是：给定 M的SRAM大小，我们可以加载大小为 $\Theta(M)$的$\mathbf{K},\mathbf{V}$ 块（算法1第6行）。对于每个$\mathbf{K}$ 和$\mathbf{V}$块，我们迭代所有$\mathbf{Q}$ 块（算法1第8行）来计算中间值，这导致了对$\mathbf{Q}$进行了 $\Theta(NdM^{-1})$次遍历。每次遍历加载 $\Theta(Nd)$个元素，这相当于 $\Theta(N^2d^2M^{-1})$次HBM访问。我们同样证明了标准注意力的反向传递需要 $\Theta(Nd + N^2)$次HBM访问，而FlashAttention的反向传递需要 $\Theta(N^2d^2M^{-1})$次HBM访问（附录B）。

我们证明了一个下界：当计算精确注意力时，对于所有 M （SRAM大小）的值，都不能渐近地改进HBM访问次数。

命题3。设 N 为序列长度， d 为头维度， M 为带$d \le M \le Nd$的*SRAM*大小。不存在一个算法能够在所有 M 属于范围 $[d, Nd]$的情况下，使用 $o(N^2d^2M^{-1})$ *HBM*访问次数来计算精确注意力。

该证明依赖于这样一个事实：对于 $M = \Theta(Nd)$，任何算法都必须执行 $\Omega(N^2d^2M^{-1}) = \Omega(Nd)$ HBM访问。这种在 M 的子区间上的下界在流算法文献 [88]中很常见。我们将证明基于 M 的参数化复杂度下界留作激动人心的未来工作。

我们验证了HBM访问次数是注意力运行时间的主要决定因素。在图2（左），我们看到尽管FlashAttention与标准注意力相比FLOP计数更高（由于反向传递中的重新计算），但它有远 fewer HBM访问次数，从而运行时间更快。在图2（中），我们变化FlashAttention的块大小 B_c ，这导致不同数量的HBM访问次数，并测量前向传递的运行时间。随着块大小增加，HBM访问次数减少（因为我们减少了输入的遍历次数），运行时间也减少。对于足够大的块大小（超过256），运行时间随后被其他因素（例如，算术运算）瓶颈。此外，更大的块大小将无法放入较小的SRAM大小。

3.3 扩展：块稀疏FlashAttention

我们将FlashAttention扩展至近似注意力：我们提出了块稀疏FlashAttention，其I/O复杂度比FlashAttention小，减少的比例与稀疏性成正比。

给定输入 $\mathbf{Q}$，$\mathbf{K}$，$\mathbf{V}$ $\in \mathbb{R}^{N \times d}$ 以及一个掩码矩阵 $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$，我们希望计算：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\mathbf{M}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some 块大小B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor, j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

Given a predefined block sparsity mask $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$ we can easily adapt Algorithm 1 to only compute the nonzero blocks of the attention matrix. The algorithm is identical to Algorithm 1, except we skip zero blocks. We reproduce the algorithm description in Algorithm 5 in Appendix B.

We also analyze the IO complexity of block-sparse FlashAttention.

Proposition 4. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Block-sparse* FlashAttention *(Algorithm 5) requires $\Theta(Nd + N^2d^2M^{-1}s)$ HBM accesses where s is the fraction of nonzero blocks in the block-sparsity mask.*

We see that applying block-sparsity yields a direct improvement by the sparsity to the larger term in the IO complexity. For large sequence lengths N, s is often set to $N^{-1/2}$ [11] or $N^{-1}\log N$ [3, 17, 92], resulting in $\Theta(N\sqrt{N})$ or $\Theta(N\log N)$ IO complexity. For downstream experiments, we use the fixed butterfly sparsity pattern [17], which has been shown to be able to approximate arbitrary sparsity [16].

In Fig. 2 (right), we validate that as the sparsity increases, the runtime of block-sparse FlashAttention improves proportionally. On the LRA benchmark, block-sparse FlashAttention achieves 2.8× speedup, while performing on par with standard attention (Section 4).

4 Experiments

We evaluate the impact of using FlashAttention to train Transformer models. We validate two claims about training time and model accuracy, and report attention runtime and memory benchmarks.

- **Training Speed.** FlashAttention outperforms the MLPerf 1.1 [58] speed record for BERT by 15%, and speeds up GPT-2 up to 3× over HuggingFace [87] and 1.8× over Megatron [77] over standard Transformers. FlashAttention speeds up the long-range arena (LRA) benchmark 2.4×.
- **Quality.** FlashAttention scales Transformers to longer sequences, yielding higher quality. FlashAttention trains GPT-2 with context length 4K faster than Megatron trains GPT-2 with context length 1K, while achieving 0.7 better perplexity. Modeling longer sequences yields 6.4 points of lift on two long-document classification tasks. Finally, FlashAttention yields the **first Transformer** that can achieve better-than-random performance on the challenging Path-X task (sequence length 16K), and block-sparse FlashAttention yields the **first sequence model** that we know of that can achieve better-than-random performance on Path-256 (sequence length 64K).
- **Benchmarking Attention.** We measure the runtime and memory performance of FlashAttention and block-sparse FlashAttention based on sequence length. We confirm that the memory footprint of FlashAttention scales linearly with seq. length and is up to 3× faster than standard attention for common seq. lengths (up to 2K). We confirm that runtime of block-sparse FlashAttention scales linearly in seq. length and is faster than all existing approximate attention baselines.

Additional experiment details are in Appendix E.

4.1 Faster Models with FlashAttention

BERT. FlashAttention yields the fastest single-node BERT training speed that we know of. We train a BERT-large [22] model with FlashAttention on Wikipedia. Table 1 compares our training time to the implementation from Nvidia that set the training speed record for MLPerf 1.1 [58]. Our implementation is 15% faster.

Table 1: Training time of BERT-large, starting from the same initialization provided by the MLPerf benchmark, to reach the target accuracy of 72.0% on masked language modeling. Averaged over 10 runs on 8×A100 GPUs.

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention (ours)	**17.4** ± 1.4

GPT-2. FlashAttention yields faster training times for GPT-2 [67] on the large OpenWebtext dataset [32] than the widely used HuggingFace [87] and Megatron-LM [77] implementations. Table 2 shows up to 3× end-to-end speedup compared to Huggingface and 1.7× speedup compared to Megatron-LM. FlashAttention

给定一个预定义的块稀疏掩码 $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$，我们可以轻松地调整算法1，仅计算注意力矩阵的非零块。该算法与算法1相同，只是我们跳过零块。我们在附录B中的算法5中重新生成了算法描述。

我们还分析了块稀疏FlashAttention的I/O复杂度。

命题4。设 N 为序列长度， d 为头维度， M 为带$d \le M \le Nd$的*SRAM*大小。块稀疏 FlashAttention*(算法5)*需要 $\Theta(Nd + N^2d^2M^{-1}s)$*HBM*访问，其中 s 是块稀疏掩码中非零块的比例。

我们看到，应用块稀疏性通过稀疏性直接改进了I/O复杂度中的较大项。对于较长的序列长度 N， s通常设置为$N^{-1/2}$ [11] 或 $N^{-1}\log N$ [3, 17, 92]，从而产生 $\Theta(N\sqrt{N})$或 $\Theta(N\log N)$的I/O复杂度。对于下游实验，我们使用固定蝶形稀疏模式 [17], ，该模式已被证明能够近似任意稀疏性 [16]。

在图2（右）中，我们验证了随着稀疏性的增加，块稀疏FlashAttention的运行时成比例地提高。在LRA基准测试中，块稀疏FlashAttention实现了2.8× 的加速比，同时其表现与标准注意力相当（第4节）。

4 个实验

我们评估了使用FlashAttention训练Transformer模型的影响。我们验证了关于训练时间和模型准确率的两个命题，并报告了注意力运行时间和内存基准测试结果。

- **训练速度**。 FlashAttention比MLPerf 1.1 [58] 速度记录快15%，并将GPT-2加速到 3× 倍于HuggingFace [87] ，以及比Megatron快8× 倍，比标准Transformer快1。8× 倍。FlashAttention将长程竞技场（LRA）基准测试加速了2.4×。
- **质量**。 FlashAttention将Transformer扩展到更长的序列，从而获得更高的质量。FlashAt- tention用4K的上下文长度训练GPT-2比Megatron用1K的上下文长度训练GPT-2快，同时实现了0.7更好的困惑度。建模更长的序列使两个长文档分类任务提升了6.4分。最后，FlashAttention产生了第一个能在具有挑战性的Path-X任务（序列长度16K）上实现优于随机性能的Transformer，而块稀疏FlashAttention产生了我们知道的第一个能在Path-256（序列长度64K）上实现优于随机性能的序列模型。
- **基准测试注意力**。 我们基于序列长度测量了FlashAttention和块稀疏FlashAttention的运行时和内存性能。我们确认FlashAttention的内存占用随序列长度线性扩展，在常见序列长度（最高2K）上比标准注意力快高达 3× 倍。我们确认块稀疏FlashAttention的运行时随序列长度线性扩展，并且比所有现有的近似注意力基线更快。

附加实验细节在附录 E 中。

4.1 使用FlashAttention的更快模型

BERT。 FlashAttention是我们所知的单节点BERT训练速度最快的实现。我们在维基百科上使用FlashAttention训练了一个BERT-large [22]模型。表1将我们的训练时间与Nvidia的记录MLPerf 1.1 [58]训练速度的实现在相同初始化下进行了比较。我们的实现速度快了15%。

表1：BERT-large的训练时间，从MLPerf基准提供的相同初始化开始，达到72.0%的掩码语言建模目标准确率。在 8×A100 GPU上运行10次取平均值。

BERT实现	训练时间（分钟）
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention（我们）	**17.4** ± 1.4

GPT-2。 FlashAttention 为 GPT-2 [67] 在大型 OpenWebtext 数据集 [32]上带来了更快的训练时间，优于广泛使用的 HuggingFace [87] 和 Megatron-LM [77] 实现。表 2 显示，与 Huggingface 相比，其端到端加速比高达 3× ，与 Megatron-LM 相比，加速比为 1.7× 。FlashAttention

achieves the same perplexity as the other two implementations, as we do not change the model definition. Appendix E includes plots of the validation perplexity throughout training, confirming that FlashAttention is as numerically stable as the baselines and produces the same training / validation curves.

Table 2: GPT-2 small and medium using FlashAttention achieve up to 3× speed up compared to Huggingface implementation and up to 1.7× compared to Megatron-LM. Training time reported on 8×A100s GPUs.

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

Long-range Arena. We compare vanilla Transformer (with either standard implementation or FlashAttention) on the long-range arena (LRA [80]) benchmark. We measure accuracy, throughput, and training time of all models. Each task has a different sequence length varying between 1024 and 4096. We follow the implementation and experimental setting in Tay et al. [80]and Xiong et al. [90].[3] Table 3 shows that FlashAttention achieves up 2.4× speed-up compared to standard attention. Block-sparse FlashAttention is faster than all of the approximate attention methods that we have tested.

Table 3: The performance of standard attention, FlashAttention, block-sparse FlashAttention, and approximate attention baselines on the Long-Range-Arena benchmarks.

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 Better Models with Longer Sequences

Language Modeling with Long Context. The runtime and memory-efficiency of FlashAttention allow us to increase the context length of GPT-2 by 4× while still running faster than the optimized implementation from Megatron-LM. Table 4 shows that that GPT-2 with FlashAttention and context length 4K is still 30% faster than GPT-2 from Megatron with context length 1K, while achieving 0.7 better perplexity.

Table 4: GPT-2 small with FlashAttention, with 4× larger context length compared to Megatron-LM, is still 30% faster while achieving 0.7 better perplexity. Training time on 8×A100 GPUs is reported.

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

Long Document Classification. Training Transformers with longer sequences with FlashAttention improves performance on the MIMIC-III [47] and ECtHR [6, 7] datasets. MIMIC-III contains intensive care unit patient discharge summaries, each annotated with multiple labels. ECtHR contains legal cases from the

[3]LRA accuracy results are known to be highly dependent on the tuning procedure [90]. Our reproduced baselines perform better than as reported in the original comparison [80].

在困惑度方面与另外两种实现相同，因为我们没有改变模型定义。附录 E 包含了整个训练过程中的验证困惑度图，证实 FlashAttention 在数值稳定性上与基线相当，并产生了相同的训练/验证曲线。表 2：GPT-2 小型和中型使用 FlashAttention，与 Huggingface 实现相比，加速比高达 3× ，与 Megatron-LM 相比，加速比为 1.7× 。训练时间报告基于 8× A100s GPU。

模型实现	OpenWebText (ppl)	训练时间（加速比）
GPT-2小 - Huggingface [87]	18.2	9.5天 (1.0×)
GPT-2小 - Megatron-LM [77]	18.2	4.7天（2.0×）
GPT-2小 - FlashAttention	18.2	**2.7天（3.5×）**
GPT-2中型 - Huggingface [87]	14.2	21.0天（1.0×）
GPT-2中型 - Megatron-LM [77]	14.3	11.5天（1.8×）
GPT-2中型 - FlashAttention	14.3	**6.9天（3.0×）**

长程竞技场。 我们在长程竞技场（LRA [80]）基准上比较了纯Transformer（使用标准实现或FlashAttention）。我们测量了所有模型的准确率、吞吐量和训练时间。每个任务具有不同的序列长度，范围在1024到4096之间。我们遵循Tay等人. [80]和Xiong等人. [90].[3] 中的实现和实验设置。表3显示，与标准注意力相比，FlashAttention实现了高达2.4× 的速度提升。块稀疏FlashAttention比我们测试的所有近似注意力方法都快。

表3：标准注意力、FlashAttention、块稀疏FlashAttention和近似注意力基线在长程竞技场基准上的性能。

模型	ListOps	Text	检索	图像	Pathfinder	Avg	加速比
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
块稀疏FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8**×
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
线性注意力 [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
局部注意力 [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 更长序列的 BetterModels

长上下文语言建模。 FlashAttention 的运行时和内存效率使我们能够在仍比 Megatron-LM 的优化实现运行更快的条件下，将 GPT-2 的上下文长度增加 4× 。表4显示，使用 FlashAttention 且上下文长度为4K 的 GPT-2 仍然比上下文长度为1K的 Megatron 中的 GPT-2 快30%，同时实现了0.7更好的困惑度。

表4：与 Megatron-LM 相比，使用 FlashAttention 的 GPT-2小 且上下文长度更大，仍然快30% 并实现了0.7更好的困惑度。报告了在 8× A100 GPU 上的训练时间。

模型实现	上下文长度	OpenWebText (ppl)	训练时间 (加速比)
GPT-2小 - Megatron-LM	1k	18.2	4.7天 (1.0×)
GPT-2小 - FlashAttention	1k	18.2	**2.7天（1.7×）**
GPT-2小 - FlashAttention	2k	17.6	3.0天（1.6×）
GPT-2小 - FlashAttention	4k	**17.5**	3.6天（1.3×）

长文档分类。 使用FlashAttention训练具有更长序列的Transformer可以提高在MIMIC-III [47] 和ECtHR [6, 7] 数据集上的性能。MIMIC-III包含重症监护病房患者出院记录，每份记录都标注了多个标签。ECtHR包含来自欧洲人权法院的法律案件，每一起案件都映射到据称被侵犯的人权公约条款。这两个数据集都包含非常长的文本文档；MIMIC中的平均token数量为2,395个，而最长的文档包含14,562个token，而ECtHR的平均值和最大值分别为2,197和49,392。我们评估了从增加预训练RoBERTa模型的序列长度中获得的提升（我们重复位置嵌入，如Beltagy等人所述）。

[3]LRA准确率结果已知高度依赖于调优流程 [90]。我们复现的基线表现优于原始比较中报告的数值 [80]。

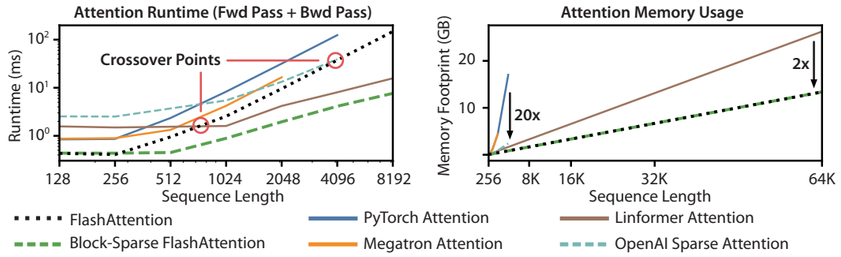


Figure 3: **Left:** runtime of forward pass + backward pass. **Right:** attention memory usage.

European Court of Human Rights, each of which is mapped to articles of the Convention of Human Rights that were allegedly violaged. Both of these datasets contain very long text documents; the average number of tokens in MIMIC is 2,395 tokens, and the longest document contains 14,562 tokens, while the average and longest numbers in ECtHR are 2,197 and 49,392, respectively. We evaluate lift from increasing the sequence length of a pretrained RoBERTa model [56] (we repeat the positional embeddings, as in Beltagy et al. [3]).

Table 5 shows that sequence length 16K outperforms length 512 by 4.3 points on MIMIC, and that length 8K outperforms length 512 by 8.5 points on ECtHR. The discrepancies may be due to subtle distribution shifts: MIMIC-III contains specialized medical text and thus may be more susceptible to a distribution shift in the document length, whereas ECtHR contains general language.

Table 5: Long Document performance (micro F_1) at different sequence lengths using FlashAttention.

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
Block-sparse FlashAttention	56.0	**63.1**

Path-X and Path-256. The Path-X and Path-256 benchmarks are challenging tasks from the long-range arena benchmark designed to test long context. The task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them, and the images are fed to the transformer one pixel at a time. In prior work, all transformer models have either run out of memory, or only achieved random performance [80]. There has been a search for alternative architectures that can model such long context [37]. We present here the first result of Transformer models being able to solve Path-X and Path-256 (Table 6). We pretrain a transformer on Path-64, and then transfer to Path-X by spatially interpolating the positional embeddings. FlashAttention achieves 61.4 accuracy on Path-X. Additionally, block-sparse FlashAttention enables the Transformers to scale to sequence length 64K, achieving 63.1 accuracy[4] on Path-256.

4.3 Benchmarking Attention

We vary sequence length and measure runtime and memory usage of FlashAttention and block-sparse FlashAttention against various attention baselines on one A100 GPU with 40 GB HBM, with dropout and a padding mask. We compare against reference implementations for exact attention, approximate attention, and sparse attention. We report a subset of baselines in the main body; Appendix E contains more baselines and full details.

[4]Path-256 requires longer sequences but has relatively shorter paths than Path-X, so it is easier to obtain a higher accuracy.

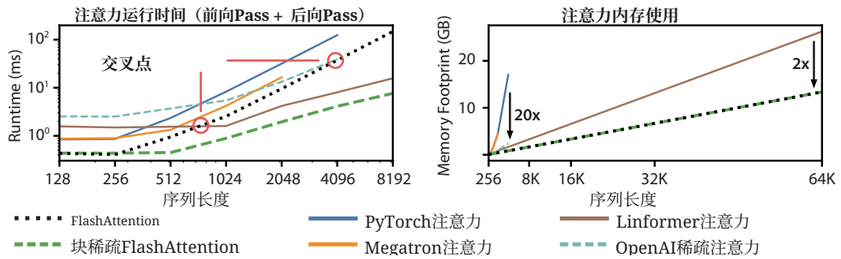


图3：**左：** + 前向传递后向传递的运行时。**右：** 注意力内存使用。

欧洲人权法院，每一起案件都映射到据称被侵犯的人权公约条款。这两个数据集都包含非常长的文本文档；MIMIC中的平均token数量为2,395个，而最长的文档包含14,562个token，而ECtHR的平均值和最大值分别为2,197和49,392，分别。我们评估了从增加预训练RoBERTa模型 [56]的序列长度中获得的提升（我们重复位置嵌入，如Beltagy等人 [3]所述）。

表5显示，序列长度16K在MIMIC上比512长4.3分，而8K在ECtHR上比512长8.5分。这种差异可能是由于细微的分布偏移：MIMIC-III包含专业医疗文本，因此可能更容易受到文档长度分布偏移的影响，而ECtHR包含通用语言。

表5：使用FlashAttention在不同序列长度下进行长文档性能测试（微观 F_1）。

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

表6：我们报告了第一个在Path-X和Path-256上能实现非随机性能的Transformer模型。

模型	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
线性注意力 [50]	✗	✗
Performer [12]	✗	✗
局部注意力 [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
块稀疏FlashAttention	56.0	**63.1**

Path-X和Path-256。 The Path-X和Path-256基准测试是长程竞技场基准测试中的具有挑战性的任务，旨在测试长上下文。该任务是对黑白色 128×128 (或 256×256)图像中的两点是否存在连接它们的路径进行分类，图像被逐像素输入Transformer。在先前的工作中，所有Transformer模型要么耗尽内存，要么仅达到随机性能 [80]。一直在寻找能够建模这种长上下文的替代架构 [37]。我们在这里展示了首个能够解决Path-X和Path-256的Transformer模型的结果（表6）。我们在Path-64上预训练一个Transformer，然后通过空间插值位置嵌入迁移到Path-X。FlashAttention在Path-X上实现了61.4的准确率。此外，块稀疏FlashAttention使Transformer能够扩展到序列长度64K，在Path-256上实现了63.1的准确率[4] 。

4.3 注意力基准测试

我们变化序列长度，测量FlashAttention和块稀疏FlashAttention在单个配备40 GB HBM的A100 GPU上相对于各种注意力基线的运行时和内存使用情况，同时使用Dropout和填充掩码。我们将其与精确注意力、近似注意力和稀疏注意力的参考实现进行比较。我们在正文中报告了部分基线；附录E包含更多基线和完整细节。

[4]Path-256需要更长的序列，但相较于Path-X路径相对较短，因此更容易获得更高的准确率。

Runtime. Figure 3 (left) reports the runtime in milliseconds of the forward + backward pass of FlashAttention and block-sparse FlashAttention compared to the baselines in exact, approximate, and sparse attention (exact numbers in Appendix E). Runtime grows quadratically with sequence length, but FlashAttention runs significantly faster than **exact attention** baselines, up to 3× faster than the PyTorch implementation. The runtimes of many approximate/sparse attention mechanisms grow linearly with sequence length, but FlashAttention still runs faster than approximate and sparse attention for short sequences due to fewer memory accesses. The **approximate attention** runtimes begin to cross over with FlashAttention at sequences between 512 and 1024. On the other hand, block-sparse FlashAttention is faster than all implementations of exact, sparse, and approximate attention that we know of, across all sequence lengths.

Memory Footprint. Figure 3 (right) shows the memory footprint of FlashAttention and block-sparse FlashAttention compared to various exact, approximate, and sparse attention baselines. FlashAttention and block-sparse FlashAttention have the same memory footprint, which grows linearly with sequence length. FlashAttention is up to 20× more memory efficient than **exact attention** baselines, and is more memory-efficient than the **approximate attention** baselines. All other algorithms except for Linformer run out of memory on an A100 GPU before 64K, and FlashAttention is still 2× more efficient than Linformer.

5 Limitations and Future Directions

We discuss limitations of our approach and future directions. Related work is given in Appendix A.

Compiling to CUDA. Our current approach to building IO-aware implementations of attention requires writing a new CUDA kernel for each new attention implementation. This requires writing the attention algorithm in a considerably lower-level language than PyTorch, and requires significant engineering effort. Implementations may also not be transferrable across GPU architectures. These limitations suggest the need for a method that supports writing attention algorithms in a high-level language (e.g., PyTorch), and compiling to IO-aware implementations in CUDA—similar to efforts such as Halide in image processing [70].

IO-Aware Deep Learning. We believe that the IO-aware approach can extend beyond attention. Attention is the most memory-intensive computation in Transformers, but every layer in a deep network touches GPU HBM. We hope our work inspires IO-aware implementations of additional modules. We discuss these potential extensions in Appendix D.

Multi-GPU IO-Aware Methods. Our IO-aware implementation of attention is optimal within constants for computing attention on a single GPU. However, the attention computation may be parallelizable across multiple GPUs [72]. Using multiple GPUs adds an additional layer to IO analysis—accounting for data transfer between GPUs. We hope our work inspires future work in this direction.

Acknowledgments

Our implementation uses Apex's FMHA code (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`) as a starting point. We thank Young-Jun Ko for the in-depth explanation of his FMHA implementation and for his thoughtful answers to our questions about CUDA. We thank Sabri Eyuboglu, Megan Leszczynski, Laurel Orr, Yuhuai Wu, Beidi Chen, and Xun Huang for their constructive feedback and suggestions on early drafts of the paper. We thank Markus Rabe and Charles Staats for helpful discussion of their attention algorithm.

We gratefully acknowledge the support of NIH under No. U54EB020405 (Mobilize), NSF under Nos. CCF1763315 (Beyond Sparsity), CCF1563078 (Volume to Velocity), and 1937301 (RTML); ARL under No. W911NF-21-2-0251 (Interactive Human-AI Teaming); ONR under No. N000141712266 (Unifying Weak Supervision); ONR N00014-20-1-2480: Understanding and Applying Non-Euclidean Geometry in Machine Learning; N000142012275 (NEPTUNE); NXP, Xilinx, LETI-CEA, Intel, IBM, Microsoft, NEC, Toshiba, TSMC, ARM, Hitachi, BASF, Accenture, Ericsson, Qualcomm, Analog Devices, Google Cloud, Salesforce, Total, the HAI-GCP & HAI-Azure Cloud Credits for Research program, the Stanford Data Science Initiative (SDSI), Department of Defense (DoD) through the National Defense Science and Engineering Graduate Fellowship (NDSEG) Program, and members of the Stanford DAWN project: Facebook, Google, and VMWare. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes

运行时。 图3（左）报告了FlashAttention和块稀疏FlashAttention在精确、近似和稀疏注意力（精确数值见附录E）中的基线相对于的前向 + 后向传递的运行时（毫秒）。运行时随序列长度呈二次方增长，但FlashAttention比**精确注意力**基线运行速度显著更快，最高比PyTorch实现快 3× 倍。许多近似/稀疏注意力机制的运行时随序列长度呈线性增长，但由于内存访问更少，FlashAttention在短序列中仍然比近似和稀疏注意力运行得更快。**近似注意力** 的运行时在512到1024个序列之间开始与FlashAttention交叉。另一方面，块稀疏FlashAttention在所有序列长度上，都比我们已知的精确、稀疏和近似注意力的所有实现更快。

内存占用。 图3（右）展示了FlashAttention和块稀疏FlashAttention与各种精确、近似和稀疏注意力基线的内存占用对比。FlashAttention和块稀疏FlashAttention具有相同的内存占用，该占用随序列长度线性增长。FlashAttention比**精确注意力** 基线最多节省 20× 的内存，并且比**近似注意力** 基线更节省内存。除Linformer外，所有其他算法在A100 GPU上运行到64K之前都会耗尽内存，而FlashAttention仍然比Linformer高效 2× 倍。

5 限制与未来方向

我们讨论l 我们方法的局限性以及未来的方向。相关工作在附录A中给出。 .

编译到CUDA。 我们当前的构建IO感知注意力实现的方法需要为每个新的注意力实现编写一个新的CUDA内核。这需要用比PyTorch低得多的语言来编写注意力算法，并且需要大量的工程工作。实现也可能无法在不同的GPU架构之间迁移。这些限制表明需要一种支持用高级语言（例如PyTorch）编写注意力算法，并编译成CUDA的IO感知实现的方法——类似于图像处理中的Halide等努力 [70]。

IO感知深度学习。 我们相信，IO感知方法可以超越注意力。注意力是Transformer中最内存密集的计算，但深度网络中的每一层都会触及GPU HBM。我们希望我们的工作能启发其他模块的IO感知实现。我们在附录D中讨论了这些潜在扩展。

多GPU IO感知方法。 我们的IO感知注意力实现是在单个GPU上计算注意力时的最优解。然而，注意力计算可以在多个GPU之间并行化 [72]。使用多个GPU为IO分析增加了一层复杂性——需要考虑GPU之间的数据传输。我们希望我们的工作能启发未来在这个方向上的研究。

致谢

我们的实现以Apex的FMHA代码（ https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha）作为起点。我们感谢Young-Jun Ko对其FMHA实现的深入解释，以及他对我们关于CUDA的问题所提供的周到回答。我们感谢Sabri Eyuboglu、Megan Leszczynski、Laurel Orr、Yuhuai Wu、Beidi Chen和Xun Huang对论文早期草稿的建设性反馈和建议。我们感谢Markus Rabe和Charles Staats对其注意力算法的有益讨论。

我们衷心感谢美国国立卫生研究院（NIH）在U54EB020405（动员）项目下的支持，美国国家科学基金会（NSF）在CCF1763315（超越稀疏性）、CCF1563078（从容量到速度）和1937301（实时机器学习）项目下的支持；美国陆军研究实验室（ARL）在W911NF-21-2-0251（人机协同）项目下的支持；美国海军研究办公室（ONR）在N000141712266（统一弱监督）项目下的支持；ONR N00014-20-1-2480项目：理解和应用机器学习中的非欧几里得几何；N000142012275（NEPTUNE）项目；NXP、Xilinx、LETI-CEA、英特尔、IBM、微软、NEC、东芝、台积电、ARM、日立、巴斯夫、埃森哲、爱立信、高通、亚德诺半导体、谷歌云、Salesforce、道达尔、HAI-GCP & HAI-Azure 研究云积分计划、斯坦福数据科学计划（SDSI）、国防部（DoD）通过国家国防科学与工程研究生奖学金（NDSEG）计划提供支持，以及斯坦福 DAWN 项目的成员：Facebook、谷歌和VMWare。美国政府被授权为政府目的复制和分发印刷品

notwithstanding any copyright notation thereon. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views, policies, or endorsements, either expressed or implied, of NIH, ONR, or the U.S. Government. Atri Rudra's research is supported by NSF grant CCF-1763481.

References

[1] Alok Aggarwal and S Vitter, Jeffrey. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.

[2] Irwan Bello. LambdaNetworks: Modeling long-range interactions without attention. *arXiv preprint arXiv:2102.08602*, 2021.

[3] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[4] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, et al. An updated set of basic linear algebra subprograms (blas). *ACM Transactions on Mathematical Software*, 28(2):135–151, 2002.

[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[6] Ilias Chalkidis, Ion Androutsopoulos, and Nikolaos Aletras. Neural legal judgment prediction in English. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4317–4323, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1424. URL `https://www.aclweb.org/anthology/P19-1424`.

[7] Ilias Chalkidis, Manos Fergadiotis, Dimitrios Tsarapatsanis, Nikolaos Aletras, Ion Androutsopoulos, and Prodromos Malakasiotis. Paragraph-level rationale extraction through regularization: A case study on european court of human rights cases. In *Proceedings of the Annual Conference of the North American Chapter of the Association for Computational Linguistics*, Mexico City, Mexico, 2021. Association for Computational Linguistics.

[8] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021. URL `http://jmlr.org/papers/v22/20-275.html`.

[9] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra, and Christopher Ré. Scatterbrain: Unifying sparse and low-rank attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.

[11] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[12] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2020.

[13] Xiang Dai, Ilias Chalkidis, Sune Darkner, and Desmond Elliott. Revisiting transformer-based models for long document classification. *arXiv preprint arXiv:2204.06683*, 2022.

[14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

无论其上是否有版权声明。本材料中表达的任何意见、发现、结论或建议均属作者所有，不一定反映美国国立卫生研究院、美国海军研究办公室或美国政府（明示或暗示）的观点、政策或认可。Atri Rudra的研究由NSF资助项目CCF-1763481支持。

参考文献

[1] Alok Aggarwal和S Vitter, Jeffrey. 排序和相关问题的输入/输出复杂度.*ACM*通讯, 31(9):1116–1127, 1988.[2] Irwan Bello. LambdaNetworks: 无需注意力机制的长程交互建模. *arXiv*预印本 *arXiv:2102.08602*, 2021.[3] Iz Beltagy, Matthew E Peters,和Arman Cohan. Longformer: 长文档Transformer. *arXiv*预印本 *arXiv:2004.05150*, 2020.[4] L Susan Blackford, Antoine Petitet, Roldan Pozo, Karin Remington, R Clint Whaley, James Demmel, Jack Dongarra, Iain Duff, Sven Hammarling, Greg Henry, 等. 基本线性代数子程序集(BLAS)的更新版本. *ACM*数学软件汇刊, 28(2):135–151, 2002.[5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, 等. 语言模型是少样本学习器.神经信息处理系统进展, 33:1877–1901, 2020.[6] Ilias Chalkidis, Ion Androutsopoulos,和Nikolaos Aletras. 英语神经法律判决预测. 在计算语言学协会第*57*届年会论文集, 页码4317–4323, 佛罗伦萨, 意大利 , 2019. 计算语言学协会 . doi: 10.18653/v1/P19-1424. URL https://www.aclweb.org/anthology/P19-1424.[7] Ilias Chalkidis, Manos Fergadiotis, Dimitrios Tsarapatsanis, Nikolaos Aletras, Ion Androutsopoulos,和Prodromos Malakasiotis. 通过正则化进行段落级推理提取: 欧洲人权法院案例研究. 在 北美计算语言学协会年会论文集, 墨西哥城, 墨西哥, 2021. 计算语言学协会.[8] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunès, François-David Collin,和Ghislain Durif. GPU上的内核操作, 带自动微分, 无内存溢出. 机器学习研究杂志, 22(74):1–6, 2021. URL http://jmlr.org/papers/v22/20-275.html.[9] Beidi Chen, Tri Dao, Eric Winsor, Zhao Song, Atri Rudra,和Christopher Ré. Scatterbrain: 统一稀疏和低秩注意力. 在 神经信息处理系统进展 *(NeurIPS)*, 2021.[10] Tianqi Chen, Bing Xu, Chiyuan Zhang,和Carlos Guestrin. 带亚线性内存成本的深度网络训练. *arXiv*预印本 *arXiv:1604.06174*, 2016.[11] Rewon Child, Scott Gray, Alec Radford,和Ilya Sutskever. 带稀疏Transformer的长序列生成. *arXiv*预印本 *arXiv:1904.10509*, 2019.[12] Krzysztof Marcin Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, Lukasz Kaiser, 等. 带Performer的注意力机制再思考. 在 学习表示国际会议 *(ICLR)*, 2020.[13] Xiang Dai, Ilias Chalkidis, Sune Darkner,和Desmond Elliott. 长文档分类的基于Transformer模型再思考. *arXiv*预印本 *arXiv:2204.06683*, 2022.[14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le,和Ruslan Salakhutdinov. Transformer-XL: 超越固定长度上下文的注意力语言模型. 在 计算语言学协会第*57*届年会论文集, 页码2978–2988, 2019.

[15] Tri Dao, Albert Gu, Matthew Eichhorn, Atri Rudra, and Christopher Ré. Learning fast algorithms for linear transforms using butterfly factorizations. In *International Conference on Machine Learning (ICML)*, 2019.

[16] Tri Dao, Nimit Sohoni, Albert Gu, Matthew Eichhorn, Amit Blonder, Megan Leszczynski, Atri Rudra, and Christopher Ré. Kaleidoscope: An efficient, learnable representation for all structured linear maps. In *International Conference on Learning Representations (ICLR)*, 2020.

[17] Tri Dao, Beidi Chen, Kaizhao Liang, Jiaming Yang, Zhao Song, Atri Rudra, and Christopher Ré. Pixelated butterfly: Simple and efficient sparse training for neural network models. In *International Conference on Learning Representations (ICLR)*, 2022.

[18] Tri Dao, Beidi Chen, Nimit Sohoni, Arjun Desai, Michael Poli, Jessica Grogan, Alexander Liu, Aniruddh Rao, Atri Rudra, and Christopher Ré. Monarch: Expressive structured matrices for efficient and accurate training. In *International Conference on Machine Learning (ICML)*, 2022.

[19] Giannis Daras, Nikita Kitaev, Augustus Odena, and Alexandros G Dimakis. Smyrf-efficient attention using asymmetric clustering. *Advances in Neural Information Processing Systems*, 33:6476–6489, 2020.

[20] Christopher De Sa, Albert Gu, Rohan Puttagunta, Christopher Ré, and Atri Rudra. A two-pronged progress in structured dense matrix vector multiplication. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1060–1079. SIAM, 2018.

[21] Peter J Denning. The working set model for program behavior. *Communications of the ACM*, 11(5): 323–333, 1968.

[22] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. 2019.

[23] Xin Dong, Shangyu Chen, and Sinno Jialin Pan. Learning to prune deep neural networks via layer-wise optimal brain surgeon. *arXiv preprint arXiv:1705.07565*, 2017.

[24] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2020.

[25] Y Eidelman and I Gohberg. On a new class of structured matrices. *Integral Equations and Operator Theory*, 34(3):293–324, 1999.

[26] Jean Feydy, Joan Glaunès, Benjamin Charlier, and Michael Bronstein. Fast geometric learning with symbolic matrices. *Advances in Neural Information Processing Systems*, 33, 2020.

[27] Jörg Flum and Martin Grohe. *Parameterized Complexity Theory*. Springer, 2006.

[28] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2018.

[29] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M Roy, and Michael Carbin. Stabilizing the lottery ticket hypothesis. *arXiv preprint arXiv:1903.01611*, 2019.

[30] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *International Conference on Machine Learning*, pages 3259–3269. PMLR, 2020.

[31] Karan Goel, Albert Gu, Chris Donahue, and Christopher Ré. It's raw! audio generation with state-space models. In *International Conference on Machine Learning (ICML)*, 2022.

[32] Aaron Gokaslan, Vanya Cohen, Pavlick Ellie, and Stefanie Tellex. Openwebtext corpus, 2019.

[15] Tri Dao，Albert Gu，Matthew Eichhorn，Atri Rudra，以及Christopher Ré。使用蝶形分解学习快速线性变换算法。在机器学习国际会议（*ICML*），2019年.[16] Tri Dao，Nimit Sohoni，Albert Gu，Matthew Eichhorn，Amit Blonder，Megan Leszczynski，Atri Rudra，以及Christopher Ré。万花筒：一种高效、可学习的所有结构化线性映射表示。在学习表示国际会议（*ICLR*），2020年.[17] Tri Dao，Beidi Chen，Kaizhao Liang，Jiaming Yang，Zhao Song，Atri Rudra，以及Christopher Ré。像素化蝶形：神经网络模型的简单高效稀疏训练。在学习表示国际会议（*ICLR*），2022年.[18] Tri Dao，Beidi Chen，Nimit Sohoni，Arjun Desai，Michael Poli，Jessica Grogan，Alexander Liu，Aniruddh Rao，Atri Rudra，以及Christopher Ré。君主：用于高效和准确训练的表达性结构矩阵。在机器学习国际会议（*ICML*），2022年.[19] Giannis Daras，Nikita Kitaev，Augustus Odena，以及Alexandros G Dimakis。使用非对称聚类的高效注意力机制。神经信息处理系统进展，33:6476–6489，2020年.[20] Christopher De Sa，Albert Gu，Rohan Puttagunta，Christopher Ré，以及Atri Rudra。结构化密集矩阵向量乘法的双重进展。在第*29*届*ACM-SIAM*离散算法年会的论文集，第1060–1079页。SIAM，2018年.[21] Peter J Denning。程序行为的-working set模型。*ACM*通讯，11(5): 323–333，1968年.[22] Jacob Devlin，Ming-Wei Chang，Kenton Lee，以及Kristina Toutanova。BERT：为语言理解而预训练深度双向Transformer。2019年.[23] Xin Dong，Shangyu Chen，以及Sinno Jialin Pan。通过逐层最优脑外科医生学习剪枝深度神经网络。*arXiv*预印本*arXiv:1705.07565*，2017年.[24] Alexey Dosovitskiy，Lucas Beyer，Alexander Kolesnikov，Dirk Weissenborn，Xiaohua Zhai，Thomas Unterthiner，Mostafa Dehghani，Matthias Minderer，Georg Heigold，Sylvain Gelly，等人。一幅图像值16x16个词：用于大规模图像识别的Transformer。在学习表示国际会议，2020年.[25] Y Eidelman和I Gohberg。关于一类新的结构化矩阵。积分方程与算子理论，34(3):293–324，1999年.[26] Jean Feydy，Joan Glaunès，Benjamin Charlier，以及Michael Bronstein。使用符号矩阵的快速几何学习。神经信息处理系统进展，33，2020年.[27] Jörg Flum和Martin Grohe。参数化复杂度理论。Springer，2006年.[28] Jonathan Frankle和Michael Carbin。彩票假设：寻找稀疏、可训练的神经网络。在学习表示国际会议，2018年.[29] Jonathan Frankle，Gintare Karolina Dziugaite，Daniel M Roy，以及Michael Carbin。稳定彩票假设。*arXiv*预印本*arXiv:1903.01611*，2019年.[30] Jonathan Frankle，Gintare Karolina Dziugaite，Daniel Roy，以及Michael Carbin。线性模式连接性和彩票假设。在机器学习国际会议，第3259–3269页。PMLR，2020年.[31] Karan Goel，Albert Gu，Chris Donahue，以及Christopher Ré。它很原始！使用状态空间模型的音频生成。在机器学习国际会议（*ICML*），2022年.[32] Aaron Gokaslan，Vanya Cohen，Pavlick Ellie，以及Stefanie Tellex。Openwebtext语料库，2019年。

[33] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data mining and knowledge discovery*, 1(1):29–53, 1997.

[34] Andreas Griewank and Andrea Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2008.

[35] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. Hippo: Recurrent memory with optimal polynomial projections. In *Advances in neural information processing systems (NeurIPS)*, 2020.

[36] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in Neural Information Processing Systems*, 34, 2021.

[37] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *The International Conference on Learning Representations (ICLR)*, 2022.

[38] Song Han, Jeff Pool, John Tran, and William J Dally. Learning both weights and connections for efficient neural networks. *arXiv preprint arXiv:1506.02626*, 2015.

[39] Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.

[40] John Hennessy and David Patterson. Memory hierarchy design. *Computer Architecture: A Quantitative Approach*, pages 390–525, 2003.

[41] Sara Hooker. The hardware lottery. *arXiv preprint arXiv:2009.06489*, 2020.

[42] Weizhe Hua, Zihang Dai, Hanxiao Liu, and Quoc V Le. Transformer quality in linear time. *arXiv preprint arXiv:2202.10447*, 2022.

[43] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. *Proceedings of Machine Learning and Systems*, 3: 711–732, 2021.

[44] Zhe Jia and Peter Van Sandt. Dissecting the Ampere GPU architecture via microbenchmarking. GPU Technology Conference, 2021.

[45] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P Scarpazza. Dissecting the nvidia Volta GPU architecture via microbenchmarking. *arXiv preprint arXiv:1804.06826*, 2018.

[46] Zhe Jia, Blake Tillman, Marco Maggioni, and Daniele Paolo Scarpazza. Dissecting the graphcore IPU architecture via microbenchmarking. *arXiv preprint arXiv:1912.03413*, 2019.

[47] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. Mimic-iii, a freely accessible critical care database. *Scientific data*, 3(1):1–9, 2016.

[48] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.

[49] Thomas Kailath, Sun-Yuan Kung, and Martin Morf. Displacement ranks of matrices and linear equations. *Journal of Mathematical Analysis and Applications*, 68(2):395–407, 1979.

[50] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, pages 5156–5165. PMLR, 2020.

[33] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. 数据立方体：一种关系聚合操作，推广了 group-by、交叉表和子总计。 数据挖掘与知识发现，1(1):29–53，1997.[34] Andreas Griewank 和 Andrea Walther. 求导数：算法微分的原理与技术. SIAM, 2008.[35] Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, 和 Christopher Ré. Hippo：具有最优多项式投影的循环内存。在 神经信息处理系统进展（*NeurIPS*），2020.[36] Albert Gu, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, 和 Christopher Ré. 结合循环、卷积和连续时间模型与线性状态空间层。 神经信息处理系统进展，34，2021.[37] Albert Gu, Karan Goel, 和 Christopher Ré. 使用结构化状态空间高效建模长序列。在 学习表示国际会议（*ICLR*），2022.[38] Song Han, Jeff Pool, John Tran, 和 William J Dally. 学习权重和连接以实现高效的神经网络。 *arXiv* 预印本 *arXiv:1506.02626*，2015.[39] Song Han, Huizi Mao, 和 William J Dally. 深度压缩：使用剪枝、训练量化和中断编码压缩深度神经网络。在 学习表示国际会议，2016.[40] John Hennessy 和 David Patterson. 内存层次结构设计。 计算机体系结构：定量方法，页码 390–525，2003.[41] Sara Hooker. 硬件彩票。 *arXiv* 预印本 *arXiv:2009.06489*，2020.[42] Weizhe Hua, Zihang Dai, Hanxiao Liu, 和 Quoc V Le. 线性时间内 Transformer 质量。 *arXiv* 预印本 *arXiv:2202.10447*，2022.[43] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, 和 Torsten Hoefler. 数据移动就是你所需要的：对 Transformer 优化的一例研究。 机器学习与系统会议录，3: 711–732，2021.[44] Zhe Jia 和 Peter Van Sandt. 通过微基准测试剖析 Ampere GPU 架构。GPU 技术大会，2021.[45] Zhe Jia, Marco Maggioni, Benjamin Staiger, 和 Daniele P Scarpazza. 通过微基准测试剖析 nvidia Volta GPU 架构。 *arXiv* 预印本 *arXiv:1804.06826*，2018.[46] Zhe Jia, Blake Tillman, Marco Maggioni, 和 Daniele Paolo Scarpazza. 通过微基准测试剖析 Graphcore IPU 架构。 *arXiv* 预印本 *arXiv:1912.03413*，2019.[47] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, 和 Roger G Mark. Mimic-III，一个免费访问的危重监护数据库。 科学数据，3(1):1–9，2016.[48] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, 等。张量处理单元的数据中心性能分析。在 第 *44* 届国际计算机体系结构研讨会会议录，页码 1–12，2017.[49] Thomas Kailath, Sun-Yuan Kung, 和 Martin Morf. 矩阵和线性方程的位移秩。 数学分析与应用杂志，68(2):395–407，1979.[50] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, 和 François Fleuret. Transformer 是 RNN：具有线性注意力的快速自回归 Transformer。在 机器学习国际会议，页码 5156–5165。PMLR，2020.

13

[51] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *The International Conference on Machine Learning (ICML)*, 2020.

[52] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite BEDRT for self-supervised learning of language representations. In *The International Conference on Learning Representations (ICLR)*, 2020.

[53] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):708–727, 2020.

[54] Valerii Likhosherstov, Krzysztof Choromanski, Jared Davis, Xingyou Song, and Adrian Weller. Sub-linear memory: How to make performers slim. *arXiv preprint arXiv:2012.11346*, 2020.

[55] Ji Lin, Yongming Rao, Jiwen Lu, and Jie Zhou. Runtime neural pruning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[56] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[57] Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *Advances in Neural Information Processing Systems*, 34, 2021.

[58] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, et al. Mlperf training benchmark. *Proceedings of Machine Learning and Systems*, 2:336–349, 2020.

[59] Frank McSherry, Michael Isard, and Derek G Murray. Scalability! but at what {COST}? In *15th Workshop on Hot Topics in Operating Systems (HotOS XV)*, 2015.

[60] Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018.

[61] NVIDIA. Nvidia Tesla V100 GPU architecture, 2017.

[62] NVIDIA. Nvidia A100 tensor core GPU architecture, 2020.

[63] NVIDIA. Nvidia H100 tensor core GPU architecture, 2022.

[64] D Stott Parker. Random butterfly transformations with applications in computational linear algebra. 1995.

[65] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[66] Markus N Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021.

[67] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[68] Jack Rae and Ali Razavi. Do transformers need deep long-range memory? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, July 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.acl-main.672`.

[69] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. In *The International Conference on Learning Representations (ICLR)*, 2020.

[51] Nikita Kitaev、Łukasz Kaiser 和 Anselm Levskaya。Reformer：高效的 Transformer。在 机器学习国际会议 *(ICML)*，2020 年.[52] Zhenzhong Lan、Mingda Chen、Sebastian Goodman、Kevin Gimpel、Piyush Sharma 和 Radu Soricut。Albert：一个轻量级的 BEDRT 用于自监督学习语言表示。在 学习表示国际会议 *(ICLR)*，2020 年.[53] Mingzhen Li、Yi Liu、Xiaoyan Liu、Qingxiao Sun、Xin You、Hailong Yang、Zhongzhi Luan、Lin Gan、Guangwen Yang 和 Depei Qian。深度学习编译器：一项综合调查。*IEEE* 并行与分布式系统汇刊，32(3):708–727，2020 年.[54] Valerii Likhosherstov、Krzysztof Choromanski、Jared Davis、Xingyou Song 和 Adrian Weller。亚线性内存：如何让 Performer 瘦削。*arXiv* 预印本 *arXiv:2012.11346*，2020 年.[55] Ji Lin、Yongming Rao、Jiwen Lu 和 Jie Zhou。运行时神经剪枝。在 I. Guyon、U. V. Luxburg、S. Bengio、H. Wallach、R. Fergus、S. Vishwanathan 和 R. Garnett 编辑的 神经信息处理系统进展，第 30 卷。Curran Associates, Inc.，2017 年.[56] Yinhan Liu、Myle Ott、Naman Goyal、Jingfei Du、Mandar Joshi、Danqi Chen、Omer Levy、Mike Lewis、Luke Zettlemoyer 和 Veselin Stoyanov。Roberta：一种鲁棒优化的 BERT 预训练方法。*arXiv* 预印本 *arXiv:1907.11692*，2019 年.[57] Xuezhe Ma、Xiang Kong、Sinong Wang、Chunting Zhou、Jonathan May、Hao Ma 和 Luke Zettlemoyer。Luna：线性统一嵌套注意力。神经信息处理系统进展，34，2021 年.[58] Peter Mattson、Christine Cheng、Gregory Diamos、Cody Coleman、Paulius Micikevicius、David Patterson、Hanlin Tang、Gu-Yeon Wei、Peter Bailis、Victor Bittorf 等人。Mlperf 训练基准。机器学习和系统会议录，2:336–349，2020 年.[59] Frank McSherry、Michael Isard 和 Derek G Murray。可扩展性！但成本是多少？在 第 *15* 届操作系统热点问题研讨会 *(HotOS XV)*，2015 年.[60] Maxim Milakov 和 Natalia Gimelshein。softmax 在线归一化计算。*arXiv* 预印本 *arXiv:1805.02867*，2018 年.[61] NVIDIA。Nvidia Tesla V100 GPU 架构，2017 年.[62] NVIDIA。Nvidia A100 张量核心 GPU 架构，2020 年.[63] NVIDIA。Nvidia H100 张量核心 GPU 架构，2022 年.[64] D Stott Parker。随机蝶形变换及其在计算线性代数中的应用。1995 年.[65] Adam Paszke、Sam Gross、Francisco Massa、Adam Lerer、James Bradbury、Gregory Chanan、Trevor Killeen、Zeming Lin、Natalia Gimelshein、Luca Antiga 等人。PyTorch：一种命令式风格、高性能的深度学习库。神经信息处理系统进展，32，2019 年.[66] Markus N Rabe 和 Charles Staats。自注意力不需要 $O(n^2)$ 内存。*arXiv* 预印本 *arXiv:2112.05682*，2021 年.[67] Alec Radford、Jeffrey Wu、Rewon Child、David Luan、Dario Amodei、Ilya Sutskever 等人。语言模型是无监督的多任务学习器。*OpenAI* 博客，1(8):9，2019 年.[68] Jack Rae 和 Ali Razavi。Transformer 需要深度长程记忆吗？在 计算语言学协会第 *58* 届年会会议录，在线，2020 年 7 月。计算语言学协会。URL https://www.aclweb.org/anthology/2020.acl-main.672.[69] Jack W Rae、Anna Potapenko、Siddhant M Jayakumar 和 Timothy P Lillicrap。用于长程序列建模的压缩 Transformer。在 学习表示国际会议 *(ICLR)*，2020 年.

[70] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

[71] Raghu Ramakrishnan, Johannes Gehrke, and Johannes Gehrke. *Database management systems*, volume 3. McGraw-Hill New York, 2003.

[72] Benjamin Recht and Christopher Ré. Parallel stochastic gradient algorithms for large-scale matrix completion. *Mathematical Programming Computation*, 5(2):201–226, 2013.

[73] Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *Advances in Neural Information Processing Systems*, 34, 2021.

[74] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.

[75] Amit Sabne. XLA: Compiling machine learning for peak performance. 2020.

[76] Victor Sanh, Thomas Wolf, and Alexander M Rush. Movement pruning: Adaptive sparsity by fine-tuning. *arXiv preprint arXiv:2005.07683*, 2020.

[77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[78] Vikas Sindhwani, Tara Sainath, and Sanjiv Kumar. Structured transforms for small-footprint deep learning. In *Advances in Neural Information Processing Systems*, pages 3088–3096, 2015.

[79] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, and Armand Joulin. Adaptive attention span in transformers. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2019.

[80] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena: A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2020.

[81] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.

[82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[83] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, and Furu Wei. Deepnet: Scaling transformers to 1,000 layers. *arXiv preprint arXiv:2203.00555*, 2022.

[84] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

[85] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[86] Michael E Wolf and Monica S Lam. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 30–44, 1991.

[70] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, 和 Saman Amarasinghe. Halide: 一种用于优化图像处理管道中的并行性、局部性和重新计算的语言和编译器。 *Acm Sigplan Notices*, 48(6):519–530, 2013.[71] Raghu Ramakrishnan, Johannes Gehrke, 和 Johannes Gehrke. *Database management systems*, 卷 3. McGraw-Hill New York, 2003.[72] Benjamin Recht 和 Christopher Ré. 大规模矩阵补全的并行随机梯度算法。 *Mathematical Programming Computation*, 5(2):201–226, 2013.[73] Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, 和 Bo Dai. Combiner: 具有稀疏计算成本的完整注意力 Transformer。 *Advances in Neural InformationProcessing Systems*, 34, 2021.[74] Aurko Roy, Mohammad Saffar, Ashish Vaswani, 和 David Grangier. 基于内容的稀疏注意力与路由 Transformer。 *Transactions of the Association for Computational Linguistics*, 9: 53–68, 2021.[75] Amit Sabne. XLA: 为峰值性能编译机器学习。 2020.[76] Victor Sanh, Thomas Wolf, 和 Alexander M Rush. Movement Pruning: 通过微调实现自适应稀疏性。 *arXiv preprint arXiv:2005.07683*, 2020.[77] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, 和 Bryan Catanzaro. Megatron-LM: 使用模型并行性训练数十亿参数的语言模型。 *arXiv preprintarXiv:1909.08053*, 2019.[78] Vikas Sindhwani, Tara Sainath, 和 Sanjiv Kumar. 小型深度学习的结构化变换。 In *Advances in Neural Information Processing Systems*, 页面 3088–3096, 2015.[79] Sainbayar Sukhbaatar, Edouard Grave, Piotr Bojanowski, 和 Armand Joulin. Transformer 的自适应注意力跨度。 In*Proceedings of the Annual Meeting of the Association for Computational Linguistics*, 2019.[80] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, 和 Donald Metzler. Long range arena: 一种高效 Transformer 的基准。 In *International Conference on Learning Representations*, 2020.[81] Yi Tay, Mostafa Dehghani, Dara Bahri, 和 Donald Metzler. 高效 Transformer: 一篇调查。 *arXivpreprint arXiv:2009.06732*, 2020.[82] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, 和 Illia Polosukhin. 注意力就是全部。 *Advances in neural information processingsystems*, 30, 2017.[83] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Dongdong Zhang, 和 Furu Wei. Deepnet: 将 Transformer 扩展到 1,000 层。 *arXiv preprint arXiv:2203.00555*, 2022.[84] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, 和 Hao Ma. Linformer: 具有线性复杂度的自注意力。 *arXiv preprint arXiv:2006.04768*, 2020.[85] Samuel Williams, Andrew Waterman, 和 David Patterson. Roofline: 一种面向多核架构的洞察性能模型。 *Communications of the ACM*, 52(4):65–76, 2009.[86] Michael E Wolf 和 Monica S Lam. 一种数据局部性优化算法。 In *Proceedings of the ACMSIGPLAN 1991 conference on Programming language design and implementation*, 页面 30–44, 1991.

[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL `https://www.aclweb.org/anthology/2020.emnlp-demos.6`.

[88] David P Woodruff. Optimal space lower bounds for all frequency moments. In *SODA*, volume 4, pages 167–175. Citeseer, 2004.

[89] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, and Michael Auli. Pay less attention with lightweight and dynamic convolutions. In *The International Conference on Learning Representations (ICLR)*, 2019.

[90] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, and Vikas Singh. Nyströmformer: A nyström-based algorithm for approximating self-attention. In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, volume 35, page 14138, 2021.

[91] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, and Shuicheng Yan. Tokens-to-token vit: Training vision transformers from scratch on imagenet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 558–567, 2021.

[92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. Big bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems*, 33, 2020.

[93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, and Josh Susskind. An attention free transformer. *arXiv preprint arXiv:2105.14103*, 2021.

[94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, and Bryan Catanzaro. Long-short transformer: Efficient transformers for language and vision. *Advances in Neural Information Processing Systems*, 34, 2021.

[87] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, 和 Alexander M. Rush. Transformers: 最先进的自然语言处理。 In*Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: SystemDemonstrations*, 页面 38–45, 在线, 2020 年 10 月. Association for Computational Linguistics. URL https://www.aclweb.org/anthology/2020.emnlp-demos.6.[88] David P Woodruff. 所有频率矩的最优空间下界。 In *SODA*, 卷 4, 页面 167–175. Citeseer, 2004.[89] Felix Wu, Angela Fan, Alexei Baevski, Yann N Dauphin, 和 Michael Auli. 使用轻量级和动态卷积减少注意力。 In *The International Conference on Learning Representations(ICLR)*, 2019.[90] Yunyang Xiong, Zhanpeng Zeng, Rudrasis Chakraborty, Mingxing Tan, Glenn Fung, Yin Li, 和 Vikas Singh. Nyströmformer: 一种基于 Nyström 的算法，用于近似自注意力。 In *Proceedings of the AAAI Conference on Artificial Intelligence. AAAI Conference on Artificial Intelligence*, 卷 35, 页面 14138, 2021.[91] Li Yuan, Yunpeng Chen, Tao Wang, Weihao Yu, Yujun Shi, Zi-Hang Jiang, Francis EH Tay, Jiashi Feng, 和 Shuicheng Yan. Tokens-to-token vit: 从零开始训练图像识的视觉 Transformer。 In*Proceedings of the IEEE/CVF International Conference on Computer Vision*, 页面 558–567, 2021.[92] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, 等. Big bird: 用于更长序列的 Transformer。 *Advances in Neural Information Processing Systems*, 33, 2020.[93] Shuangfei Zhai, Walter Talbott, Nitish Srivastava, Chen Huang, Hanlin Goh, Ruixiang Zhang, 和 Josh Susskind. 无注意力的 Transformer。 *arXiv preprint arXiv:2105.14103*, 2021.[94] Chen Zhu, Wei Ping, Chaowei Xiao, Mohammad Shoeybi, Tom Goldstein, Anima Anandkumar, 和 Bryan Catanzaro. Long-short Transformer: 用于语言和视觉的高效 Transformer。 *Advances inNeural Information Processing Systems*, 34, 2021.

16

A Related Work

IO-Aware Runtime Optimization. The broad concept of optimizing for reading and writing to fast/slow memory has a long history in computer science and has been known by many names. We draw the most direct connection to the literature of analyzing I/O complexity in this work [1], but concepts of memory hierarchies are fundamental and has appeared in many forms, from the working set model [21], to data locality [86], to the Roofline model of arithmetic intensity [85], to analyses of scalability [59], to standard textbook treatments of computer architecture [40]. We hope that this work encourages the community to adopt these ideas in more parts of the deep learning stack.

Efficient ML Models with Structured Matrices. Matrix multiply is the core computational bottleneck of most machine learning models. To reduce the computational complexity, there have been numerous approaches to learn over a more efficient set of matrices. These matrices are called *structured matrices*, which have subquadratic ($o(n^2)$ for dimension $n \times n$) number of parameters and runtime. Most common examples of structured matrices are sparse and low-rank matrices, along with fast transforms commonly encountered in signal processing (Fourier, Chebyshev, sine/cosine, orthogonal polynomials). There have been several more general classes of structured matrices proposed in machine learning: Toeplitz-like [78], low-displacement rank [49], quasi-separable [25]). The butterfly pattern we use for our block-sparse attention is motivated by the fact that butterfly matrices [15, 64] and their products have been shown to be able to express any structured matrices with almost optimal runtime and number of parameters [16, 20]. However, even though structured matrices are efficient in theory, they have not seen wide adoption since it is hard to translate their efficiency to wall-clock speedup since dense unconstrained matrix multiply has very optimize implementation, a phenomenon known as the hardware lottery [41]. Extensions of butterfly matrices [17, 18] aimed to make butterfly matrices more hardware-friendly.

Sparse Training. Our block-sparse FlashAttention can be seen as a step towards making sparse model training more efficient. Sparse models have seen success in compressing models for inference (pruning) by sparsifying the weight matrices [23, 38, 39, 55, 76]. For model training, the lottery tickets hypothesis [28, 29, 30] suggests that there are a set of small sub-networks derived from a larger dense network that performs as well as the original dense network. Out block-sparse FlashAttention can also be seen as a fixed lottery ticket in the context of attention: we fix the sparsity pattern to be the butterfly pattern through training, and observe that it performs almost as well as the (dense) FlashAttention on the Long-range Arena tasks.

Efficient Transformer. Transformer-based models have become the most widely-used architecture in natural language processing [22] and computer vision [24, 91]. However, one of their computational bottlenecks is that their time and memory scales quadratic in the sequence length. There are numerous approaches to overcome this bottleneck, including approximation with hashing (i.e., sparse) such as Reformer [51] and Smyrf [19] and with low-rank approximation such as Performer [12, 54]. One can even combine sparse and low-rank approximation for better accuracy (e.g., Longformer [3], BigBird [92], Scatterbrain [9], Long-short transformer [94], Combiner [73]). Other approaches include compressing along the sequence dimension to attend to multiple tokens at once [52, 57, 79, 89]. One can also attend over the states from previous sequences to help lengthen the context (e.g., Transformer-XL [14] and Compressive Transformer [69]). We recommend the survey [81] for more details.

There are several lines of work on developing other modules instead of attention to model longer context. HiPPO [35] and its extensions, most notably S4 [31, 36, 37] projects the history on a polynomial basis, allowing accurate reconstruction of the history through state-space models. They combine the strengths of CNNs (efficient training), RNNs (efficient inference), and continuous models (robust to change in sampling rates). LambdaNetworks [2], AFT [93] and FLASH [42] are other attempts at replacing attention in the context of image classification and language modeling.

B Algorithm Details

We first derive the forward and backward passes of attention and show that they can be computed in a memory-efficient manner (requiring extra memory linear instead of quadratic in the sequence length). Though they reduce the amount of extra memory required, naively they still incur quadratic HBM accesses, resulting in slower execution speed. We describe the FlashAttention algorithm to implement both the forward

相关工作

IO感知运行时优化优化读写快/慢内存的广泛概念在计算机科学中有着悠久的历史，并以许多不同的名称为人所知。我们在本文中与分析I/O复杂度的文献建立了最直接的联系 [1], ，但内存层次结构的概念是基础性的，并以多种形式出现，从工作集模型 [21], 到数据局部性 [86], ，再到算术强度 [85], 的Roofline模型，到可扩展性 [59], 的分析，再到计算机体系结构的标准教科书处理 [40]。我们希望这项工作能鼓励社区将这些思想应用于深度学习堆栈的更多部分。

具有结构化矩阵的高效机器学习模型。 矩阵乘法是大多数机器学习模型的核心计算瓶颈。为了降低计算复杂度，已经有许多方法来学习更高效的一组矩阵。这些矩阵被称为 结构化矩阵，它们具有次平方（$o(n^2)$ 对于维度 $n \times n$）数量的参数和运行时。结构化矩阵最常见的例子是稀疏矩阵和低秩矩阵，以及信号处理中常见的快速变换（傅里叶变换、切比雪夫变换、正弦/余弦变换、正交多项式）。在机器学习中已经提出了几种更通用的结构化矩阵类别：Toeplitz类 [78], 低位移秩 [49], 准可分 [25]) 。我们用于块稀疏注意力的蝶形模式，其动机是蝶形矩阵 [15, 64] 及其乘积已被证明能够以几乎最优的运行时和参数数量 [16, 20]来表达任何结构化矩阵。然而，尽管结构化矩阵在理论上很高效，但由于很难将其效率转化为实际的加速比，因为密集无约束矩阵乘法具有非常优化的实现，这种现象被称为硬件彩票 [41]。蝶形矩阵 [17, 18] 的扩展旨在使蝶形矩阵更易于硬件处理。

稀疏训练。 我们的块稀疏FlashAttention可以看作是使稀疏模型训练更高效的一步。稀疏模型在通过稀疏化权重矩阵 [23, 38, 39, 55, 76]压缩推理（剪枝）模型方面取得了成功。对于模型训练，彩票票假说 [28, 29, 30]表明，存在一组从更大的密集网络中派生出来的小型子网络，其性能与原始密集网络相当。我们的块稀疏FlashAttention也可以看作是在注意力上下文中的一个固定彩票票：我们通过训练将稀疏模式固定为蝶形模式，并观察到它在长程竞技场任务上的表现几乎与（密集）FlashAttention一样好。

高效 Transformer。 基于 Transformer 的模型已成为自然语言处理 [22]和计算机视觉 [24, 91]中最广泛使用的架构。然而，它们的一个计算瓶颈是它们的时间和内存随序列长度呈二次方扩展。有大量方法来克服这个瓶颈，包括使用哈希（即稀疏）的近似方法，例如 Reformer [51] 和 Smyrf [19] ，以及使用低秩近似方法，例如 Performer [12, 54]。甚至可以结合稀疏和低秩近似以获得更好的准确率（例如，Longformer [3],BigBird [92],Scatterbrain [9],Long-short transformer [94], Combiner [73]) 。其他方法包括沿着序列维度进行压缩，以便一次关注多个标记 [52, 57, 79, 89]。还可以关注先前序列的状态以帮助延长上下文（例如，Transformer-XL [14] 和 Compressive Transformer [69]) 。我们推荐阅读调查报告 [81] 以获取更多细节。

在模型处理更长上下文方面，除了注意力之外，还有几条研究线在开发其他模块。HiPPO [35] 及其扩展，尤其是 S4 [31, 36, 37], 将历史信息投影到多项式基上，通过状态空间模型实现历史的精确重建。它们结合了CNN（高效训练）、RNN（高效推理）和连续模型（对采样率变化具有鲁棒性）的优势。LambdaNetworks [2], AFT [93]和 FLASH [42]是在图像分类和语言建模的上下文中替代注意力的其他尝试。

B算法细节

我们首先推导出注意力的正向和反向传递，并证明它们可以以内存高效的方式计算（所需额外内存与序列长度呈线性关系而非二次方关系）。尽管它们减少了所需的额外内存量，但就其本身而言，它们仍然会导致二次方HBM访问，从而降低执行速度。我们描述了FlashAttention算法，该算法在GPU上实现正向和反向传递，减少HBM访问，从而实现更快的运行时和更小的内存占用。

and the backward passes on GPUs that reduces HBM accesses, leading to both faster runtime and smaller memory footprint.

B.1 Memory-efficient forward pass

The main challenge in making attention memory-efficient is the softmax that couples the columns of $\mathbf{K}$ (and columns of $\mathbf{V}$). Our approach is to compute the softmax normalization constant separately to decouple the columns. This technique [60] has been used in the literature [51, 66] to show that attention computation does not need quadratic *extra* memory (though the number of HBM accesses is still quadratic, resulting in slow run-time).

For simplicity, we omit here the max-shifting step during softmax. The full algorithm in Appendix B.3 contains all the steps.

Recall that given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

We have that $S_{ij} = q_i^T k_j$ where q_i and k_j are the i-th and j-th columns of $\mathbf{Q}$ and $\mathbf{K}$ respectively. Define the normalization constants of softmax:

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

Let v_j be the j-th column of $\mathbf{V}$, then the i-th columns of the output is

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

We see that once L_i is computed, we can compute o_i without extra memory by repeatedly summing $\frac{e^{q_i^T k_j}}{L_i} v_j$. Therefore the forward pass can be computed with $O(n)$ extra memory:

1. Compute L_i for all i according to Eq. (1), which takes $O(n)$ extra memory.
2. Compute o_i for all i according to Eq. (2), which takes $O(d)$ extra memory.

B.2 Memory-efficient backward pass

We derive the backward pass of attention and show that it can also be computed with linear memory. Rabe and Staats [66] suggests that the backward pass can be done without quadratic extra memory by applying gradient checkpointing to the memory-efficient forward pass. We instead derive the backward pass explicitly and show how it can be computed in a memory-efficient manner.

Suppose that there is a scalar loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dO}$ denotes $\frac{\partial \phi}{\partial \mathbf{O}}$). We want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ denote $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$ respectively).

The gradient $\mathbf{dV}$ is easy to see. Applying reverse-mode autodiff by hand (aka the chain rule), we obtain (in matrix notation) $\mathbf{dV} = \mathbf{P}^T \mathbf{dO}$. Thus:

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

Since we already computed L_i, dv_j can be computed without extra memory by repeated summing.

The gradients $\mathbf{dQ}$ and $\mathbf{dK}$ are a little more complicated. We go through the gradients $\mathbf{dP}$ and $\mathbf{dS}$ first. From Eq. (2), we have that $\mathbf{dP} = \mathbf{dO}\mathbf{V}^T$, and so:

$$dP_{ij} = do_i^T v_j.$$

Recall that $P_{i:} = \mathrm{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \mathrm{softmax}(x)$ is $\mathrm{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

该算法在GPU上实现正向和反向传递，减少HBM访问，从而实现更快的运行时和更小的内存占用。

B.1 内存高效的正向传递

使注意力内存高效的主要挑战在于softmax将$\mathbf{K}$（以及$\mathbf{V}$的列）耦合在一起。我们的方法是将softmax归一化常数单独计算以解耦列。这种技术在文献 [51, 66] 中已被使用，以证明注意力计算不需要二次额外内存（尽管HBM访问次数仍然是二次的，导致运行时间缓慢）。

为简单起见，我们在此省略softmax过程中的最大偏移步骤。附录B.3中的完整算法包含所有步骤。

回想一下，给定 输入序列$\mathbf{Q}$，$\mathbf{K}$，$\mathbf{V} \in \mathbb{R}^{N \times d}$，我们希望计算注意力输出$\mathbf{O} \in \mathbb{R}^{N \times d}$ ：

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

我们有 $S_{ij} = q_i^T k_j$其中 q_i和 k_j分别是$\mathbf{Q}$和$\mathbf{K}$的第 i-列和第 j-列。定义softmax的归一化常数：

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

设 v_j 是 $\mathbf{V}$ 的第 j 列，则输出的第 i 列是

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

我们看到，一旦计算了 L_i ，我们就可以通过重复求和 $\frac{e^{q_i^T k_j}}{L_i} v_j$ 来计算 o_i 而不需要额外内存。因此，正向传递可以用 $O(n)$ 额外内存来计算：

1. 根据 Eq. (1) 计算 L_i 的所有 i ，这需要 $O(n)$ 额外内存。
2. 根据 Eq. (2) 计算 o_i 的所有 i ，这需要 $O(d)$ 额外内存。

B.2 内存高效的反向传递

我们推导了注意力的反向传递，并表明它也可以用线性内存来计算。Rabe 和 Staats [66] 建议通过将梯度检查点应用于内存高效的正向传递，可以在没有二次额外内存的情况下完成反向传递。我们相反地明确推导了反向传递，并展示了如何以内存高效的方式计算它。

假设存在一个标量损失函数 ϕ，并令输出梯度为$\mathbf{dO} \in \mathbb{R}^{n \times d}$（其中 $\mathbf{dO}$ 表示$\frac{\partial \phi}{\partial \mathbf{O}}$）。我们想要计算输入梯度 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$（其中 $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ 分别表示 $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$）。

梯度 $\mathbf{dV}$ 很容易看出。手动应用反向模式自动微分（即链式法则），我们得到（以矩阵形式表示）$\mathbf{dV} = \mathbf{P}^T \mathbf{dO}$。因此：

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

由于我们已经计算了 L_i， dv_j 可以通过重复求和来计算，而无需额外内存。

梯度 $\mathbf{dQ}$ 和 $\mathbf{dK}$ 稍微复杂一些。我们先处理梯度 $\mathbf{dP}$ 和 $\mathbf{dS}$ 。根据公式 (2)，我们有 $\mathbf{dP} = \mathbf{dO}\mathbf{V}^T$，因此：

$$dP_{ij} = do_i^T v_j.$$

回想一下 $P_{i:} = \mathrm{Softmax}(S_{i:})$。利用 $y = \mathrm{Softmax}(x)$ 的雅可比矩阵是 $\mathrm{diag}(y) - yy^T$ 这一事实，我们得到

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

where $\circ$ denotes pointwise multiplication.

Define

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

Now we can get the gradients $\mathbf{dQ}$ and $\mathbf{dK}$. Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) k_j. \tag{5}$$

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) q_i. \tag{6}$$

Therefore the backward pass can also be computed with $O(n)$ extra memory:

1. Compute dv_j for all j according to Eq. (3), which takes $O(d)$ extra memory.
2. Compute D_i for all i according to Eq. (4), which takes $O(n)$ extra memory.
3. Compute dq_i for all i according to Eq. (5), which takes $O(d)$ extra memory.
4. Compute dk_j for all j according to Eq. (6), which takes $O(d)$ extra memory.

B.3 FlashAttention: Forward Pass

We describe the full details of FlashAttention forward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

where $\tau \in \mathbb{R}$ is some softmax scaling (typically $\frac{1}{\sqrt{d}}$), MASK is some masking function that sets some entries of the input to $-\infty$ and keep other entries the same (e.g., key padding mask when sequences in the batch don't have the same lengths and are padded), and $\text{dropout}(x, p)$ applies dropout to x elementwise (i.e., output $\frac{x}{1-p}$ with probability $1-p$ and output 0 with probability p for each element x).

The full algorithm is in Algorithm 2. We save the output $\mathbf{O}$, the softmax statistics ℓ and m, and the pseudo-random number generator state $\mathcal{R}$ for the backward pass.

其中 $\circ$ 表示逐元素乘法。

定义

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

因此

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

现在我们可以得到梯度 $\mathbf{dQ}$和 $\mathbf{dK}$。回想 $S_{ij} = q_i^T k_j$，所以

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) k_j. \tag{5}$$

类似地

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) q_i. \tag{6}$$

因此反向传递也可以使用 $O(n)$ 额外内存来计算：

1. 根据 Eq. (3) 计算 dv_j 对所有 j，这需要 $O(d)$ 额外内存。
2. 根据 Eq. (4) 计算 D_i 对所有 i，这需要 $O(n)$ 的额外内存。
3. 根据 Eq. (5) 计算 dq_i 对所有 i，这需要 $O(d)$ 的额外内存。
4. 根据 Eq. (6) 计算 dk_j 对所有 j，这需要 $O(d)$ 的额外内存。

B.3 FlashAttention: 正向Pass

我们描述了 FlashAttention 正向传递的完整细节。给定输入序列 $\mathbf{Q}$，$\mathbf{K}$，$\mathbf{V} \in \mathbb{R}^{N \times d}$，我们想要计算注意力输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

其中 $\tau \in \mathbb{R}$ 是某种 softmax缩放（通常为 $\frac{1}{\sqrt{d}}$），掩码是某种掩码函数，它将输入的一些元素设置为 $-\infty$ 并保持其他元素不变（例如，当批次中的序列长度不同并进行填充时使用键填充掩码），而 $\text{dropout}(x, p)$ 对 x 进行逐元素 dropout（即，以概率 $1-p$ 输出 $\frac{x}{1-p}$，以概率 p 输出 0，对于每个元素 x）。

完整算法在算法2中。我们保存输出 $\mathbf{O}$、softmax统计l 和 m，以及反向传递的伪随机数生成器状态 R。

Algorithm 2 FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}.

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

B.4 FlashAttention: Backward Pass

We describe the full details of FlashAttention backward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, the output $\mathbf{O} \in \mathbb{R}^{N \times d}$, and the output gradient $\mathbf{dO}$, we want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$.

We first describe the standard attention backward pass in Algorithm 3 for completeness.

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dS}\mathbf{K}$, write $\mathbf{dQ}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We now make two observations about FlashAttention backward pass:

1. We do not need to store the dropout mask of size $O(N^2)$ from the forward pass. Instead, we can save the pseudo-random number generator states from the forward pass and re-generate the dropout mask in the backward pass. This allows us to only use $O(N)$ extra memory.

2. When computing the softmax gradient, we use Eq. (4) to compute $D_i = P_{i:}^\top dP_{i:}$ without reducing over $P_{i:}$ and $dP_{i:}$ of size N (they might not fit into SRAM). Instead we can rewrite $D_i = do_i^\top o_i$ and compute the dot product between vectors of size d.

算法2 FlashAttention 前向传递 需要：矩阵 Q、K、V $\in \mathbb{R}^{N \times d}$ 在 HBM 中，片上 SRAM 大小 M，softmax 缩放常数 $\tau \in \mathbb{R}$，掩码函数 mask，dropout 概率 p_{drop}。 1: 初始化伪随机数生成器状态 R 并保存到 HBM。 2: 设置块大小 $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$。 3: 初始化 O $= (0)_{N \times d} \in \mathbb{R}^{N \times d}$，l $= (0)_N \in \mathbb{R}^N$，$m = (-\infty)_N \in \mathbb{R}^N$ 在 HBM 中。 4: 将 Q 分成 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $Q_1, \dots, Q_{T_r}$，每个块大小为 $B_r \times d$，将 K、V 分成 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $K_1, \dots, K_{T_c}$ 和 $V_1, \dots, V_{T_c}$，每个块大小为 $B_c \times d$。 5: 将 O 分成 T_r 个块 $O_i \dots, O_{T_r}$，每个块大小为 $B_r \times d$，将 l 分成 T_r 个块 $l_i, \dots, l_{T_r}$，每个块大小为 B_r，将 m 分成 T_r 个块 $m_1 \dots, m_{T_r}$，每个块大小为 B_r。 6: for $1 \le j \le T_c$ do 7: 将 K_j、V_j 从 HBM 加载到片上 SRAM。 8: for $1 \le i \le T_r$ do 9: 将 Q_i、O_i、l_i, m_i 从 HBM 加载到片上 SRAM。 10: 在片上，计算 $S^{ij} = \tau Q_i K_j^T \in \mathbb{R}^{B_r \times B_c}$。 11: 在片上，计算 $S_{ij}^{\text{masked}} = \text{mask}(S^{ij})$。 12: 在片上，计算 $\tilde{m}_{ij} = \text{rowmax}(S_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$，$\tilde{P}^{ij} = \exp(S_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$(逐点)，$\tilde{l}^{ij} = \text{rowsum}(\tilde{P}^{ij}) \in \mathbb{R}^{B_r}$。 13: 在片上，计算 $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$，$l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} l_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{l}_{ij} \in \mathbb{R}^{B_r}$。 14: 在片上，计算 $\tilde{P}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{P}^{ij}, p_{\text{drop}})$。 15: 将 $O_i \leftarrow \text{diag}(l_i^{\text{new}})^{-1}(\text{diag}(l_i) e^{m_i - m_i^{\text{new}}} O_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{P}_{ij}^{\text{dropped}} V^j)$ 写入 HBM。 16: 将 $l_i \leftarrow l_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$写入 HBM。 17: end for 18: end for 19: 返回 O、l、m、R。

B.4 FlashAttention: 反向传递

我们描述了 FlashAttention 反向传递的完整细节。给定输入序列 **Q**， **K**， **V** $\in \mathbb{R}^{N \times d}$，输出 **O** $\in \mathbb{R}^{N \times d}$，以及输出梯度 **dO**，我们希望计算输入梯度 **dQ**， **dK**， **dV** $\in \mathbb{R}^{N \times d}$。我们首先在算法 3 中描述标准注意力反向传递，以保持完整性。

算法 3 标准注意力反向传递**要求：** 矩阵 **Q**， **K**， **V**， **dO** $\in \mathbb{R}^{N \times d}$、**P** $\in \mathbb{R}^{N \times N}$存储在 HBM 中。

1: 从 HBM 按块加载 P、dO，计算 dV $= P^\top dO \in \mathbb{R}^{N \times d}$，将 dV 写入 HBM。 2: 从 HBM 按块加载 dO、V，计算 dP $= dO V^\top \in \mathbb{R}^{N \times N}$，将 dP 写入 HBM。 3: 从 HBM 读取 P、dP，计算 dS $\in \mathbb{R}^{N \times N}$其中 $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$，将 dS 写入 HBM。 4: 从 HBM 按块加载 dS 和 K，计算 dQ = dSK，将 dQ 写入 HBM。 5: 从 HBM 按块加载 dS 和 Q，计算 $dK = dS^\top Q$，将 dK 写入 HBM。 6: 返回 dQ、dK、dV。

我们现在对 FlashAttention 后向传递有两个观察结果：

1. 我们不需要存储来自前向传递的 dropout mask 大小为 $O(N^2)$ 的掩码。相反，我们可以保存前向传递的伪随机数生成器状态，并在后向传递中重新生成 dropout mask。这使我们只需要使用 $O(N)$ 的额外内存。

2. 在计算softmax梯度时，我们使用公式(4)来计算 $D_i = P_{i:}^\top dP_{i:}$，不进行对大小为 N 的$P_{i:}$ 和 $dP_{i:}$ 的规约（它们可能不适合放入SRAM）。相反，我们可以重写 $D_i = do_i^\top o_i$ 并计算大小为 d的向量之间的点积。

The full FlashAttention backward pass algorithm is in Algorithm 4. Conceptually it is just a block version of the derivation in Appendix B.2.

Algorithm 4 FlashAttention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
    On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We see that similar to the forward pass, the backward pass performs $O(N^2)$ FLOPs and only requires $O(N)$ extra memory beyond inputs, output, output gradient, and input gradients.

We analyze the IO-complexity of the backward pass, similar to the forward pass (Theorem 2).

Theorem 5. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) backward pass requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *backward pass (Algorithm 4) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

The proof is in Appendix C.

完整的FlashAttention反向传递算法在算法4中。从概念上讲，它只是附录B.2中推导的块版本。

算法4 FlashAttention 反向传递

要求： 矩阵 $\mathbf{Q}$、$\mathbf{K}$、$\mathbf{V}$、$\mathbf{O}$、$\mathbf{dO} \in \mathbb{R}^{N \times d}$ 在HBM中，向量 $l m \in \mathbb{R}^N$ 在HBM中，片上SRAM大小 M，softmax缩放常数 $\tau \in \mathbb{R}$，掩码函数 mask，dropout概率 p_{drop}，伪随机数生成器状态 R 来自前向传递。

将伪随机数生成器状态设置为R。
设置块大小 $B_c = \lceil \frac{M}{4d} \rceil$、$B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$。
将 $\mathbf{Q}$ 分成 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ 个块 $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ 个大小为 $B_r \times d$ 的块，并将 $\mathbf{K}$，$\mathbf{V}$ 分成 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ 个块 $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ 以及 $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$，大小为 $B_c \times d$ 个。
将 $\mathbf{O}$ 分成 T_r 块 $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ 大小为 $B_r \times d$ 每个，将 $\mathbf{dO}$ 分成 T_r 块 $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ 大小 $B_r \times d$ 每个，将 l 分成 T_r 块 $l_i, \dots, l_{T_r}$ 大小 B_r each, 将 m 分成 T_r 块 $m_1, \dots, m_{T_r}$ 大小 B_r 每个。
在 HBM 中初始化 $\mathbf{dQ} = (0)_{N \times d}$ 并将其分成 T_r 块 $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ 大小为 $B_r \times d$ 的每个。初始化 $\mathbf{dK} = (0)_{N \times d}$，$\mathbf{dV} = (0)_{N \times d}$ 在HBM中，并将 $\mathbf{dK}$，$\mathbf{dV}$ 分成 T_c 个 $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ 和 $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$，大小为 $B_c \times d$ 个。
对于 $1 \le j \le T_c$ **进行**
  将 $\mathbf{K}_j$，$\mathbf{V}_j$ 从HBM加载到片上SRAM。
  初始化 **头维度** $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}$, $\mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ 在片上SRAM上。
  for $1 \le i \le T_r$ **do**
    从HBM加载 $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, l_i, m_i$ 到片上SRAM。
    在芯片上，计算 $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。
    在芯片上，计算 $\mathbf{S}_{ij}^{\text{masked}} = \text{mask}(\mathbf{S}_{ij})$。
    片上，计算 $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$。
    片上，计算 dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$，其中每个元素的值为 $\frac{1}{1-p_{\text{drop}}}$ ——— 以概率 $1 - p_{\text{drop}}$ 并以概率 p_{drop} 值为 0。
    片上，计算 $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$（逐元素相乘）。
    片上，计算 $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$。
    片上，计算 $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    片上，计算 $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (逐元素乘法).
    片上，计算 $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    片上，计算 $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    将 $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ 写入HBM.
    片上，计算 $\mathbf{d\tilde{K}}_j \leftarrow$ **头维度** $_j + \tau$ **头维度** $\mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$,
  结束for循环
  写入 $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ 到HBM。
结束for循环
返回 $\mathbf{dQ}$，$\mathbf{dK}$，$\mathbf{dV}$。

我们看到，与正向传递类似，反向传递执行 $O(N^2)$ FLOPs，并且仅需要在输入、输出、输出梯度以及输入梯度之外额外使用 $O(N)$ 内存。我们分析反向传递的 IO 复杂度，与正向传递类似（定理2）。

定理5. 设 N 为序列长度，d 为头维度，M 为具有 $d \le M \le Nd$ 的 *SRAM* 大小。标准注意力（算法0）反向传递需要 $\Theta(Nd + N^2)$ *HBM* 访问，而 FlashAttention反向传递（算法4）需要 $\Theta(N^2 d^2 M^{-1})$ *HBM* 访问。

证明在附录C中。

B.5 Comparison with Rabe and Staats [66]

We describe here some similarities and differences between our FlashAttention algorithm and the algorithm of Rabe and Staats [66].

Conceptually, both FlashAttention and Rabe and Staats [66] operate on blocks of the attention matrix using the well-established technique of tiling (or softmax scaling) [51, 60]. To reduce the memory footprint, both methods avoid storing the large attention matrix in the forward pass and recompute it in the backward pass.

The first major difference is that Rabe and Staats [66] focuses on the reducing the total memory footprint (maximum amount of GPU memory required) while FlashAttention focuses on reducing memory accesses (the number of memory reads/writes). As mentioned in Section 2, the amount of memory access is the primary determining factor of runtime. Reducing memory accesses also necessarily reduces the total amount of memory required (e.g., if an operation incurs A memory accesses, then its total memory requirement is at most A). As a result, FlashAttention is faster than standard attention (2-4×) while Rabe and Staats [66] is around the same speed or slightly slower than standard attention. In terms of total memory required, both methods offer substantial memory saving.

The second difference between the two methods is the way information is summarized from each block to pass to the next block. Rabe and Staats [66] summarizes each block with its temporary output along with the softmax normalization statistics. At the end of the forward pass, the temporary outputs of all the blocks are combined using the statistics to produce the final output. FlashAttention instead incrementally updates the output (Algorithm 1 line 12) after processing each block, so only one copy of the output is needed (instead of K copies for K blocks). This means that FlashAttention has smaller total memory requirement compared to Rabe and Staats [66].

The final major difference is the way the backward pass is computed. Rabe and Staats [66] uses gradient checkpointing to recompute the attention matrix and the temporary output of each block. FlashAttention instead simplifies the backward pass analytically (Appendices B.2 and B.4). It only recomputes the attention matrix and does not recompute the temporary output of each block. This reduces the memory requirement for the backward pass and yields speedup.

C Proofs

Proof of Theorem 1. We first count the number of FLOPs and extra memory required.

The dominating FLOPs are from matrix multiplication. In the inner loop, (Algorithm 1 line 9), we compute $\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ for $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We also compute (Algorithm 1 line 12) $\tilde{\mathbf{P}}_{ij}\mathbf{V}_j \in \mathbb{R}^{B_r \times d}$ for $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$ and $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We execute the inner loops $T_c T_r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$ times. Therefore the total number of FLOPs is

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d).$$

In terms of extra memory required, we see that we need $O(N)$ memory to store the statistics (ℓ, m).

We now prove the algorithm's correctness by induction on j for $0 \le j \le T_c$. Let $\mathbf{K}_{:j} \in \mathbb{R}^{jB_c \times d}$ be the first jB_c rows of $\mathbf{K}$, and similarly $\mathbf{V}_{:j} \in \mathbb{R}^{jB_c \times d}$ the the first jB_c rows of $\mathbf{V}$. Let $\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top \in \mathbb{R}^{N \times jB_c}$, and $\mathbf{P}_{:,:j} = \mathrm{softmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^{N \times jB_c}$ (softmax applied row-wise). Let $m^j, \ell^{(j)}, \mathbf{O}^{(j)}$ be the values of $m, \ell, \mathbf{O}$ in HBM after the j-th iteration of the outer loop (Algorithm 1 line 5). (Note that these values of $m, \ell, \mathbf{O}$ are updated after each iteration of the outer loop.) We want to show that after the j-th iteration of the outer loop, we have computed in HBM:

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\mathbf{V}_{:j} \in \mathbb{R}^{N \times d}.$$

Based on our initialization (Algorithm 1 line 2), this claim is true for $j = 0$ (i.e., before the any iteration of the outer loop is executed). Suppose that the claim holds for some $j = 0, \ldots, T_c - 1$. We want to show that the claim also holds for $j + 1$. Indeed, when we update the statistics in the inner loop (Algorithm 1 line 10)

B.5 与Rabe和Staats的比较 [66]

我们在此描述了我们FlashAttention算法与Rabe和Staats的算法 [66]之间的一些相似之处和不同之处。

从概念上讲，FlashAttention和Rabe和Staats [66] 都使用成熟的tiling（或softmax缩放） [51, 60]技术对注意力矩阵的块进行操作。为了减少内存占用，这两种方法都避免在前向传递中存储大型注意力矩阵，并在后向传递中重新计算它。

第一个主要区别是，Rabe和Staats [66] 专注于减少总内存占用（所需的最大GPU内存量），而FlashAttention专注于减少内存访问（内存读写次数）。如第2节所述，内存访问量是运行时的主要决定因素。减少内存访问也必然减少所需的内存总量（例如，如果操作产生 A 次内存访问，则其总内存需求最多为 A）。因此，FlashAttention比标准注意力（2-4×）更快，而Rabe和Staats [66]的速度与标准注意力相当或略慢。在所需总内存方面，这两种方法都提供了显著的内存节省。

这两种方法之间的第二个区别在于如何从每个块中汇总信息并将其传递到下一个块。Rabe和Staats [66] 使用每个块的临时输出以及softmax归一化统计来汇总每个块。在前向传递结束时，所有块的临时输出会使用这些统计信息进行组合，以产生最终输出。而FlashAttention则是在处理每个块后增量更新输出（算法1第12行），因此只需要一个输出副本（而不是为 K 块使用 K 个副本）。这意味着FlashAttention的总内存需求比Rabe和Staats [66]更小。

最后一个主要区别在于反向传递的计算方式。Rabe和Staats [66] 使用梯度检查点来重新计算注意力矩阵和每个块的临时输出。而FlashAttention则通过解析方法简化反向传递（附录B.2和B.4）。它只重新计算注意力矩阵，而不会重新计算每个块的临时输出。这减少了反向传递的内存需求，并带来了加速比。

C 证明

定理*1*的证明。我们首先计算所需的FLOPs和额外内存 .

占主导地位的FLOPs来自矩阵乘法。在内循环中，(算法1第9行)，我们计算 $\mathbf{Q}_i\mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$对于 $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ 和 $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$，这需要 $O(B_r B_c d)$ FLOPs。我们还计算 (算法1第12行) $\tilde{\mathbf{P}}_{ij}\mathbf{V}_j \in \mathbb{R}^{B_r \times d}$对于 $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$和 $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$，这需要 $O(B_r B_c d)$ FLOPs。我们执行内循环 $T_c T_r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$次。因此FLOPs总数是

— —

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d).$$

在内存方面 额外内存需求方面，我们看到我们需要 $O(N)$ 内存来存储统计信息 (l, m).

我们通过归纳法对 j进行证明，以验证算法的正确性。设$\mathrm{K}_{:j} \in \mathbb{R}^{jB_c \times d}$为K的前 jB_c行，$\mathrm{V}_{:j} \in \mathbb{R}^{jB_c \times d}$为V的前 jB_c行。令$\mathrm{S}_{:,:j} = \mathrm{Q}\mathrm{K}_{:j}^\top \in \mathbb{R}^{N \times jB_c}$， 以及$\mathrm{P}_{:,:j} = \mathrm{softmax}(\mathrm{S}_{:,:j}) \in \mathbb{R}^{N \times jB_c}$(softmax按行应用)。设 $m^j, \mathrm{l}^{(j)}, \mathrm{O}^{(j)}$ 为外循环（算法 1 第 5 行）第 j 次迭代后 HBM 中 $m, \mathrm{l}, \mathrm{O}$ 的值。（注意这些 $m, \mathrm{l}, \mathrm{O}$ 的值在每次外循环迭代后都会更新。）我们需要证明在外循环第 j次迭代后，我们在HBM中已计算：

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j}\mathbf{V}_{:j} \in \mathbb{R}^{N \times d}.$$

根据我们的初始化（算法1第2行），此命题在 $j = 0$ （即外循环的任何迭代执行之前）时成立。假设命题对于某些 $j = 0, \ldots, T_c - 1$成立。我们想要证明命题也对于 $j + 1$成立。确实，当我们更新内循环中的统计信息（算法1第10行）时

on the $(j+1)$-th iteration of the outer loop, we update $m^{(j+1)} = \max(m^{(j)}, \tilde{m})$ where $\tilde{m} \in \mathbb{R}^N$ is the row-max of $\mathbf{S}_{:,j:j+1}$, the slice of $\mathbf{S}$ from column jB_c to column $(j+1)B_c - 1$. This implies that

$$m^{(j+1)} = \text{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

Similarly, we update

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

where $\tilde{\ell} = \text{rowsum}(\exp(\mathbf{S}_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$. By the same algebraic manipulation in Section 3.1, we obtain:

$$\ell^{(j+1)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

Let $\mathbf{V}_{j:j+1}$ be the slice of $\mathbf{V}$ from column jB_c to column $(j+1)B_c - 1$, we also update:

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\text{diag}(\ell^{(j)})\exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \text{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

We then see that the claim is also true for $j+1$. By induction, the claim is true for all $j = 0, \ldots, T_c$.

When $j = T_c$, we conclude that the final value of $\mathbf{O}$ in HBM is $\text{softmax}(\mathbf{S})\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$.

□

Proof of Theorem 2. We first analyze the IO complexity of standard attention implementation. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ is written to HBM.

In the first step of computing the matrix multiply $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, the inputs $\mathbf{Q}, \mathbf{K}$ are read from HBM and the output $\mathbf{S} \in \mathbb{R}^{N \times N}$ is written to HBM (Algorithm 0 line 1). This incurs $\Theta(Nd + N^2)$ HBM accesses.

In the second step of computing $\mathbf{P} = \text{softmax}(\mathbf{S})$, the input $\mathbf{S}$ is read from HBM and the output $\mathbf{P}$ is written to HBM (Algorithm 0 line 2). This incurs $\Theta(N^2)$ HBM accesses.

In the last step of computing $\mathbf{O} = \mathbf{P}\mathbf{V}$, the inputs $\mathbf{P}, \mathbf{V}$ are read from global memory and the output $\mathbf{O}$ is written to HBM (Algorithm 0 line 3). This incurs $\Theta(Nd + N^2)$ HBM accesses.

Overall, standard attention implementation requires $\Theta(Nd + N^2)$ global memory accesses.

We now analyze the IO complexity of streaming attention.

Following Algorithm 1, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once (Algorithm 1 line 6). We make T_c passes over $\mathbf{Q}$ and $\mathbf{O}$, each pass loading all of $\mathbf{Q}$ and all of $\mathbf{O}$ to HBM (Algorithm 1 line 8). Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

We derive the conditions on the block sizes B_c and B_r. We need the blocks $\mathbf{K}_j$ and $\mathbf{V}_j$ of size $B_c \times d$ to fit into on-chip memory, which translates to:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

Similarly, we need the blocks $\mathbf{Q}_i, \mathbf{O}_i$ of size $B_r \times d$ to fit into on-chip memory, which translates to:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

Finally, we need the block $\mathbf{S}_{ij}$ of size $B_r \times B_c$ to fit into on-chip memory, which translates to:

$$B_r B_c = O(M).$$

在外循环的第（ $j+1$）次迭代中，我们更新 $m^{(j+1)}$=max($m^{(j)}$， $\tilde{m}$)，其中 $\tilde{m} \in \mathbb{R}^N$是$\mathbf{S}_{:,j:j+1}$的行最大值，即 $\mathbf{S}$从列 jB_c到列（ $j+1$)$B_c - 1$的切片。这意味着

$$m^{(j+1)} = \text{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

同样地，我们更新

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

˜其中 l =行和 (exp($\mathbf{S}_{:,j:j+1} - \tilde{m}$)) $\in \mathbb{R}^N$。通过第3.1节的相同代数操作，我们得到 in:

$$\ell^{(j+1)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

令 $\mathbf{V}_{j:j+1}$是 $\mathbf{V}$从第 jB_c列到第($j+1$)$B_c - 1$列的切片，我们也更新:

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\text{diag}(\ell^{(j)})\exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \text{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

然后我们看到该命题对 $j+1$也成立。通过归纳法，该命题对所有 $j = 0$， ...， T_c都成立。当 $j = T_c$时，我们得出结论，HBM中 $\mathbf{O}$ 的最终值是softmax(S)V = softmax(QK$^\top$)V。

□

定理2的证明。我们首先分析标准注意力实现的I/O复杂度。输入$\mathbf{Q}$， $\mathbf{K}$， $\mathbf{V} \in \mathbb{R}^{N \times d}$位于HBM中，并且在算法结束时输出 $\mathbf{O} \in \mathbb{R}^{N \times d}$被写入HBM。在计算矩阵乘法 $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$的第一步中，输入 $\mathbf{Q}$,$\mathbf{K}$ 从HBM读取，输出 $\mathbf{S} \in \mathbb{R}^{N \times N}$被写入HBM（算法0第1行）。这导致 $\Theta(Nd + N^2)$ HBM访问。

在计算<样式 id='1'>P </样式> =Softmax(<样式 id='4'>S</样式>)的第二步中，输入<样式 id='6'>S </样式>从HBM读取，输出<样式 id='8'>P </样式>写入HBM（算法0第2行）。这会导致 $\Theta(N^2)$次HBM访问。

在计算<样式 id='1'>O </样式> = <样式 id='4'>PV</样式>的最后一步中，输入<样式 id='6'>P</样式><样式 id='8'>,</样式><样式 id='10'>V </样式>从全局内存读取，输出<样式 id='12'>O </样式>写入HBM（算法0第3行）。这会导致 $\Theta(Nd + N^2)$次HBM访问。

Ov全，标准注意力实现需要 $\Theta(Nd + N^2)$次全局内存访问 .

我们现在分析流式注意力的I/O复杂度。

根据算法1，我们看到<样式 id='1'>K </样式>和<样式 id='3'>V </样式>的每个元素都从HBM加载一次（算法1第6行）。我们进行 T_c 次遍历<样式 id='6'>Q </样式>和<样式 id='8'>O</样式>，每次遍历将全部<样式 id='10'>Q </样式>和全部<样式 id='12'>O </样式>加载到HBM（算法1第8行）。因此，HBM访问的次数是 $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$。

我们推导出关于块大小 B_c 和 B_r 的条件。我们需要大小为 $B_c \times d$ 的 $\mathbf{K}^j$ 块和 $\mathbf{V}^j$ 块能够适配片上存储器，这等价于:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

类似地，w 我们需要大小为 $B_r \times d$ 的 $\mathbf{Q}_i$,$\mathbf{O}_i$ 块能够适配片上存储器，这等价于:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

最终 y，我们需要大小为 $B_r \times B_c$ 的<样式 id='1'>S</样式>^{ij} 能够适配片上存储器，这相当于 :

$$B_r B_c = O(M).$$

We therefore set:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Proof of Proposition 3. For contradiction, suppose that there exists an algorithm that computes exact attention where the number for HBM access for all $M \in [d, Nd]$ is

$$o\left(\frac{N^2d^2}{M}\right).$$

In the regime of $M = \Theta(Nd)$, this results in the number of HBM accesses:

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

However, the input to attention (matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$) and the output $\mathbf{O}$ have size Nd and they start out being in HBM, so if the algorithm computes exact attention it must incur at least $\Omega(Nd)$ HBM accesses. This is a contradiction. □

Proof of Theorem 5. The IO complexity of the attention backward is very similar to the IO complexity of the attention forward (Theorem 2). Here we provide a sketch of the proof.

We first analyze the IO complexity of standard attention backward pass. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the outputs $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ are written to HBM.

At each step of the standard attention backward pass, one needs to load inputs of size Nd or N^2 from HBM, and needs to write the outputs of size N^2 or Nd to HBM. This incurs $\Theta(Nd + N^2)$ HBM accesses.

We now analyze the IO complexity of FlashAttention backward pass.

Similar to Theorem 2, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once. Each element of $\mathbf{dK}$ and $\mathbf{dV}$ is only written to HBM once. We make T_c passes over $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$, each pass loading all of $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ to HBM. We also make T_c passes over $\mathbf{dQ}$, each pass reading/writing all of $\mathbf{dQ}$ from/to HBM. Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

As in the proof of Theorem 2, the constraints on the block sizes are that:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

因此我们设定：

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

然后我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM访问的次数是：

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

命题3的证明。为矛盾，假设存在一个计算精确注意力的算法，其中所有 $M \in [d, Nd]$ 的HBM访问次数为

$$o\left(\frac{N^2d^2}{M}\right).$$

在 $M = \Theta(Nd)$的情况下，这导致HBM访问的次数：

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

然而，注意力输入（矩阵 $\mathbf{Q}$， $\mathbf{K}$， $\mathbf{V}$）和输出 $\mathbf{O}$ 的大小为 Nd，并且它们最初位于HBM中，因此如果算法计算精确注意力，它必须至少产生 $\Omega(Nd)$次HBM访问。这是一个矛盾。□

定理5的证明。注意力的反向IO复杂度与注意力的正向IO复杂度非常相似（定理2）。在这里我们提供了一个证明的概要。

我们首先分析标准注意力反向传递的I/O复杂度。输入 $\mathbf{Q}$， $\mathbf{K}$， $\mathbf{V}$， $\mathbf{dO} \in \mathbb{R}^{N \times d}$驻留在HBM中，算法结束时输出 $\mathbf{dQ}$， $\mathbf{dK}$， $\mathbf{dV} \in \mathbb{R}^{N \times d}$ 被写入HBM。

在标准注意力反向传递的每一步，需要从HBM加载大小为 Nd 或 N^2 的输入，并将大小为 N^2 或 Nd 的输出写入HBM。这会产生 $\Theta(Nd + N^2)$ 次HBM访问。

我们现在分析FlashAttention反向传递的I/O复杂度。

类似于定理2，我们看到 $\mathbf{K}$ 和 $\mathbf{V}$ 的每个元素只从HBM加载一次。 $\mathbf{dK}$ 和 $\mathbf{dV}$ 的每个元素只写入HBM一次。我们对 $\mathbf{Q}$,$\mathbf{O}$， $\mathbf{dO}$进行 T_c 轮遍历，每轮遍历将全部的 $\mathbf{Q}$， $\mathbf{O}$， $\mathbf{dO}$加载到HBM。我们还对 $\mathbf{dQ}$进行 T_c 轮遍历，每轮遍历从/写入全部的 $\mathbf{dQ}$ 。因此HBM访问次数为 $\Theta(Nd + NdT_c) = \Theta(NdT_c)$。

如定理2的证明所示，块大小的约束条件是：

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

然后我们有：

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

因此，HBM访问次数为：

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Algorithm 5 Block-Sparse FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$, block sparsity mask $M \in \{0,1\}^{N/B_r \times N/B_c}$..

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    if $M_{ij} \neq 0$ **then**
      Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
      On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
      On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
      On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
      On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
      On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
      Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
      Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
    end if
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D Extension Details

D.1 Block-sparse FlashAttention

We describe the full block-sparse FlashAttention algorithm in Algorithm 5. The algorithm is identical to Algorithm 2, except that we skip zero blocks.

We prove the IO-complexity of block-sparse FlashAttention.

Proof of Proposition 4. The proof is very similar to the proof of Theorem 2. For the block-sparse case, notice that we only need to load blocks corresponding to nonzero blocks. As a result, the number of HBM accesses are scaled by s, the fraction of nonzero blocks in the block-sparsity mask. However, for small values of s, we would still need to write the result $\mathbf{O} \in \mathbb{R}^{N \times d}$. Therefore the number of HBM accesses is

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 Potential Extensions

We discuss here a few potential extensions of the IO-aware approach to speed up deep learning training.

Multi-GPU Attention. Large language models are trained on hundreds or thousands of GPUs, and one typically splits the attention computation between 4-8 GPUs on the same node [77]. This introduces another level of memory hierarchy: beside GPU SRAM and GPU HBM, we also have the HBM of other

算法5块稀疏FlashAttention正向传递需要：矩阵Q、K、V $\in \mathbb{R}^{N \times d}$在HBM中，片上SRAM大小为 M，softmax缩放常数 $\tau \in \mathbb{R}$，掩码函数mask，dropout概率 p_{drop}，块大小 $B_c = \lceil \frac{M}{4d} \rceil, B_r =$最小 $(\lceil \frac{M}{4d} \rceil, d)$，块稀疏掩码 $M \in \{0,1\}^{N/B_r \times N/B_c}$.. 1: 初始化伪随机数生成器状态R并保存到HBM。 2: 初始化O = $(0)_{N \times d} \in \mathbb{R}^{N \times d}$，l = $(0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$在HBM中。 3: 将Q分成 $T_r = \left\lceil \frac{N}{B_r} \right\rceil$个大小为 $B_r \times d$的块$\mathrm{Q}_1, \dots, \mathrm{Q}_{T_r}$，将 K、V 分成 $T_c = \left\lceil \frac{N}{B_c} \right\rceil$个大小为 $B_c \times d$ 的块 $\mathrm{K}_1, \dots, \mathrm{K}_{T_c}$ 和 $\mathrm{V}_1, \dots, \mathrm{V}_{T_c}$。 4: 将 O 分成 T_r 个大小为 $B_r \times d$的块$\mathrm{O}_i, \dots, \mathrm{O}_{T_r}$，将l分成 T_r个大小为 B_r的块$\mathrm{l}_i, \dots, \mathrm{l}_{T_r}$，将 m分成 T_r个大小为 B_r的块 $m_1, \dots, m_{T_r}$。 5: for $1 \le j \le T_c$ do 6: 将 K_j，V_j 从 HBM 加载到片上 SRAM。 7: for $1 \le i \le T_r$ do 8: if $M_{ij} \neq 0$ then 9: 将Q_i，O_i，l_i，m_i从HBM加载到片上SRAM。 10: 在片上，计算$\mathrm{S}_{ij} = \tau \mathrm{Q}_i \mathrm{K}_j^T \in \mathbb{R}^{B_r \times B_c}$。 11: 在片上，计算$\mathrm{S}_{ij}^{\text{masked}} = \text{mask}(\mathrm{S}_{ij})$。 12: 在片上，计算 $\tilde{m}_{ij} = \text{rowmax}(\mathrm{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$，$\tilde{\mathrm{P}}_{ij} = \exp(\mathrm{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$(逐点)，$\tilde{\mathrm{l}}_{ij} = \text{rowsum}(\tilde{\mathrm{P}}_{ij}) \in \mathbb{R}^{B_r}$。 13: 在片上，计算 $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$，$\mathrm{l}_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \mathrm{l}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathrm{l}}_{ij} \in \mathbb{R}^{B_r}$。 14: 在片上，计算 $\tilde{\mathrm{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathrm{P}}_{ij}, p_{\text{drop}})$。 15: 将$\mathrm{O}_i \leftarrow \text{diag}(\mathrm{l}_i^{\text{new}})^{-1}(\text{diag}(\mathrm{l}_i) e^{m_i - m_i^{\text{new}}} \mathrm{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathrm{P}}_{ij}^{\text{dropped}} \mathrm{V}_j)$写入HBM。 16: 将$\mathrm{l}_i \leftarrow \mathrm{l}_i^{\text{new}}$，$m_i \leftarrow m_i^{\text{new}}$写入HBM。 17: end if 18: end for 19: end for 20: 返回O，l，m，R。

D 扩展细节

D.1 块稀疏FlashAttention

我们在算法5中描述了完整的块稀疏FlashAttention算法。该算法与算法2相同，只是我们跳过了零块。

我们证明了块稀疏FlashAttention的IO复杂度。

命题*4*的证明。 证明非常类似于定理2的证明。对于块稀疏情况，请注意我们只需要加载对应于非零块的那些块。因此，HBM访问次数按 s缩放， s是块稀疏掩码中非零块的比例。然而，对于小的 s值，我们仍然需要写入结果 $\mathbf{O} \in \mathbb{R}^{N \times d}$。因此，HBM访问次数是

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 潜在扩展

我们在此讨论 IO感知方法的一些潜在扩展，以加快深度学习训练速度 .

多GPU注意力。大型语言模型通常在数百或数千个GPU上训练，并且通常将注意力计算分配到同一节点上的4-8个GPU [77]。这引入了另一个内存层次结构级别：除了GPU SRAM和GPU HBM外，我们还有其他GPU的HBM。

GPUs. For very long sequences, the different GPUs on the same node can cooperate to compute attention by taking into account the asymmetry of different levels of memory hierarchy.

Sparse MLP layers. Typical dense MLP layers are compute-bound and not memory-bound. To improve their efficiency, MLP layers with sparse weight matrices can be used [17]. However, many sparse MLP layers are instead memory-bound, and their speedup is often not proportional to the sparsity. We believe that an IO-aware implementation can alleviate this issue and realize the benefits of sparsity. We are excited about future work in this direction, to reduce the computational requirement of large models and improve their wall-block runtime.

Kernel machine learning. Our approach in FlashAttention relies on the fact that the $N \times N$ attention matrix is a function of a low-rank matrix $\mathbf{Q}\mathbf{K}^\top$ (of rank $d \ll N$). As a result, we can repeatedly load the inputs $\mathbf{Q}, \mathbf{K}$ and recompute the block of the attention matrix that we need, significantly reducing HBM access. As similar scenario happens in kernel machine learning: each element K_{ij} of the $N \times N$ kernel matrix $\mathbf{K}$ is a function of two vectors of size $d \ll N$, as it measures the similarity between two datapoints x_i and x_j. The KeOps library [8, 26] is a successful example of how reducing memory reads/writes can speed up kernel operations. We hope that this will motivate kernel methods that focus more on reducing IOs instead of just FLOPs.

E Full Experimental Results

E.1 BERT

We train BERT-large following the training procedure and hyperparameters of the reference MLPerf 1.1 implementation. In particular, we use the LAMB optimizer with learning rate 3.75e-3, with batch size 448, trained for at most 7100 steps. The training is stopped once the validation accuracy (for masked language modeling) reaches the target 72.0%, and the wall-clock run-time is measured. We train with FP16 precision using Apex AMP (with O2 optimization level).

We compare our results with the reported training speed from Nvidia that was submitted to MLPerf 1.1 (Table 1).

We use the same train / validation data split provided by MLPerf 1.1 reference implementation. In particular, we evaluate on the same 10000 validation examples as the baseline from Nvidia.

We train the model on 8×A100-80GB GPUs. Each training run takes between 16 and 19 minutes, and we average the results of 10 runs.

E.2 GPT-2

We use the standard implementations of GPT-2 [67] from Huggingface `transformers` library and from Nvidia's Megatron-LM repo. We follow the training recipe of the Megatron-LM repo.

We use an effective batch size of 512, and use gradient accumulation to fit into available GPU memory. We use the AdamW optimizer, with learning rate 6e-4 for GPT-2 small and 1.5e-4 for GPT-2 medium, and weight decay of 0.1. All models are trained with the same hyperparameters for 400K steps. We run all implementations with mixed-precision training (PyTorch AMP).

We use the Openwebtext dataset, with the GPT-2 BPE tokenizer. We randomly select 0.5% of the dataset as the validation set, with the rest being used as training set. This random selection of validation set is done once, and all models are evaluated on the same validation set.

We train the model on 8×A100-40GB GPUs, and we measure the wall-clock training time. Training GPT-2 small takes between 2.7-9.5 days, and training GPT-2 medium takes between 6.9-21.0 days (Table 2).

In Fig. 4, we plot of the validation perplexity throughout training of GPT-2 small/medium, using either HuggingFace implementation or our FlashAttention implementation. We see that FlashAttention behaves the same as the baseline implementation and the validation perplexity curves of the two implementations almost lie on top of each other.

Long Document Classification. For MIMIC-III and ECtHR, we follow the hyperparameters of Dai et al. [13].

对于非常长的序列，同一节点上的不同GPU可以通过考虑不同内存层次结构级别的非对称性来协同计算注意力。

稀疏MLP层。 典型的密集MLP层是计算受限而非内存受限。为了提高它们的效率，可以使用具有稀疏权重矩阵的MLP层 [17]。然而，许多稀疏MLP层反而是内存受限的，并且它们的加速比通常与稀疏性不成比例。我们相信，一个感知I/O的实现可以缓解这个问题，并实现稀疏性的好处。我们对这个方向上的未来工作感到兴奋，以减少大型模型的计算需求并提高它们的墙时钟运行时。

内核机器学习。 我们的FlashAttention方法依赖于这样一个事实：$N \times N$注意力矩阵是一个低秩矩阵 $\mathbf{Q}\mathbf{K}^\top$（秩为 $d \ll N$）的函数。因此，我们可以重复加载输入 $\mathbf{Q}$,$\mathbf{K}$ 并重新计算我们需要的注意力矩阵块，从而显著减少HBM访问。在内核机器学习中也会发生类似的场景：K_{ij} 内核矩阵 $\mathbf{K}$ 的每个元素都是两个大小为 $d \ll N$ 的向量的函数，因为它度量了两个数据点 x_i 和 x_j 之间的相似性。KeOps库 [8, 26] 是一个成功示例，展示了减少内存读写如何加速内核操作。我们希望这能激励更多关注减少IO而不是仅仅减少FLOPs的内核方法。

E 全部 实验结果

E.1 BERT

我们遵循参考 MLPerf 1.1 实现的训练流程和超参数来训练 BERT-large。具体来说，我们使用学习率为 3.75e-3 的 LAMB 优化器，批大小为 448，最多训练 7100 步。当掩码语言模型的验证准确率达到目标 72.0% 时停止训练，并测量墙上运行时间。我们使用 Apex AMP（O2 优化级别）以 FP16 精度进行训练。

我们将我们的结果与提交给 MLPerf 1.1 的 Nvidia 报告的训练速度进行比较（表 1）。

我们使用 MLPerf 1.1 参考实现提供的相同训练 / 验证数据分割。具体来说，我们在与 Nvidia 基线相同的 10000 个验证示例上评估。

我们在 8×A100-80GB GPU 上训练模型。每次训练运行耗时 16 到 19 分钟，我们取 10 次运行结果的平均值。

E.2 GPT-2

我们使用来自Huggingface Transformers库和Nvidia Megatron-LM仓库的GPT-2 [67] 标准实现。我们遵循Megatron-LM仓库的训练配方。

我们使用512的有效批大小，并使用梯度累积以适应可用的GPU内存。我们使用AdamW优化器，对于GPT-2小型的学习率为6e-4，对于GPT-2中型的学习率为1.5e-4，权重衰减为0.1。所有模型都使用相同的超参数训练400K步。我们使用混合精度训练（PyTorch AMP）运行所有实现。

我们使用OpenWebText数据集，以及GPT-2 BPE分词器。我们随机选择数据集的0.5%作为验证集，其余部分作为训练集。这个验证集的随机选择只进行一次，所有模型都在同一个验证集上进行评估。

我们在 8×A100-40GB GPU上训练模型，并测量了墙钟训练时间。训练GPT-2小模型需要2.7-9.5天，训练GPT-2中型模型需要6.9-21.0天（表2）。

在图4中，我们绘制了GPT-2小/中型模型在训练过程中的验证困惑度，使用了HuggingFace实现或我们的FlashAttention实现。我们看到FlashAttention表现与基线实现相同，两种实现的验证困惑度曲线几乎完全重叠。

长文档分类。 对于MIMIC-III和ECtHR，我们遵循Dai等人[13]的超参数设置。

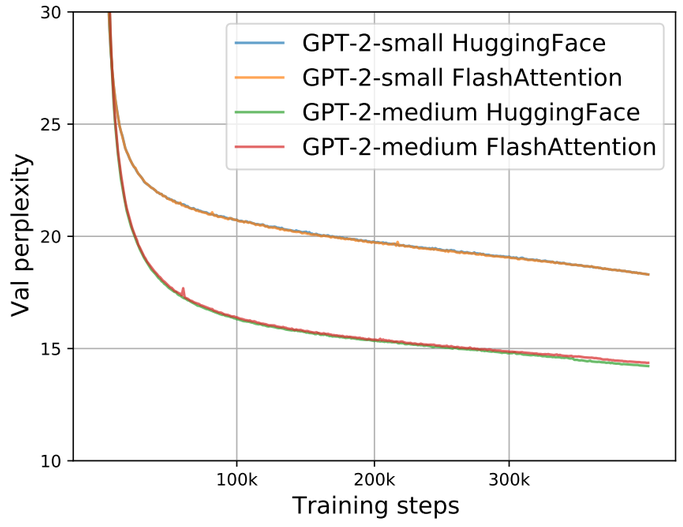


Figure 4: Validation perplexity of GPT-2 small/medium using two implementations. We confirm that FlashAttention yields the same validation curves as the baseline implementation from HuggingFace.

E.3 LRA details

We follow the hyperparameters from the Long-range arena paper [80], the Long-range arena repo (`https://github.com/google-research/long-range-arena`), and the Nyströmformer reproduction [90]. To be generous to the baseline methods, if we are unable to reproduce the performance of any baseline for any of the five tasks, we report the better performance from Tay et al. [80] or Xiong et al. [90] for that baseline on that task.

After hyperparameter tuning, almost all of the attention methods achieve similar accuracy on all of the five LRA tasks.

We run all methods with mixed-precision training, except for Performer (not stable with mixed precision) and Local Attention (implementation does not support FP16).

To calculate the overall wallclock-time speedup, we take the geometric mean of the wallclock-time speedup of each of the five tasks.

Path-X For Path-X and Path-256, we follow the hyperparameters from the PathFinder-32 experiments from the long-range arena paper[80]. For both, we first pretrain a model on Path-64. We take the checkpoint after 200 epochs, upsample its positional embedding (we duplicate the positional embeddings gridwise in space), and fine-tune it on the downstream task for 200 epochs with one epoch of linear warmup, and cosine decay of the learning rate. For Path-X, we take the best performing checkpoint (according to val accuracy), and additionally fine-tune it for 200 epochs with the same warmup and learning rate (this adds roughly 4 points of accuracy to FlashAttention for Path-X, but the model starts overfitting afterwards).

E.4 Comparison with Apex FMHA

We compare our method/implementation with Apex FMHA (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`).

When we started this project, Apex FMHA was the fastest implementation of attention (that we knew of), tailored for short sequences of length at most 512. In fact, almost all MLPerf submissions for BERT training benchmark running on Nvidia GPUs use FMHA for their model code, as of MLPerf 1.1 [58]. Since

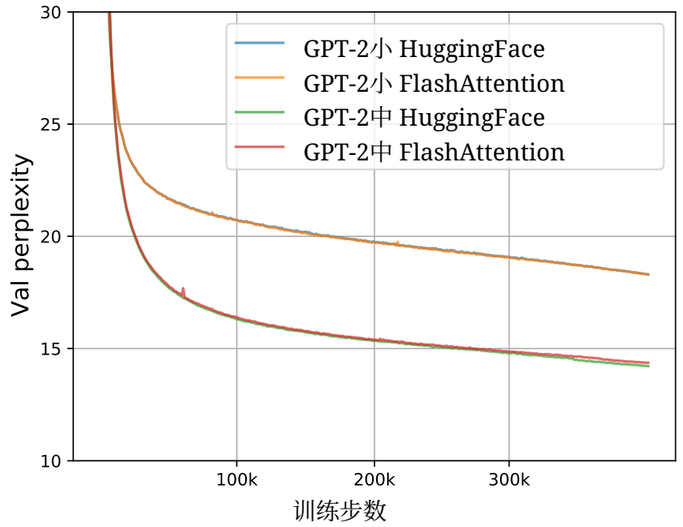


图4：使用两种实现方式验证GPT-2小/中的困惑度。我们确认FlashAttention与HuggingFace的基线实现产生了相同的验证曲线。

E.3 LRA细节

我们遵循长程竞技场论文 [80], 、长程竞技场仓库（https: //github.com/google-research/long-range-arena）以及Nyströmformer复现 [90]的超参数。为了对基线方法更加宽容，如果我们无法复现任何基线在五个任务中的性能，我们将报告Tay等人 [80] 或Xiong等人 [90] 在该基线任务上的更好性能。

在超参数调优后，几乎所有注意力方法在五个LRA任务上均达到相似的准确率。

我们使用混合精度训练运行所有方法，但Performer（混合精度不稳定）和局部注意力（实现不支持FP16）除外。

为了计算 o 总体 wallclock-time 加速比，我们取 wallclock-time 加速比的几何平均值的几何平均值 sks.

Path-X 对于 Path-X 和 Path-256，我们遵循长程竞技场论文中 PathFinder-32 实验的超参数[80]。对于这两种模型，我们首先在 Path-64 上预训练一个模型。我们取 200 个 epoch 后的检查点，对其位置嵌入进行上采样（我们在空间中网格化地复制位置嵌入），并在下游任务上进行 200 个 epoch 的微调，并使用一个 epoch 的线性预热和余弦衰减的学习率。对于 Path-X，我们取表现最佳的检查点（根据验证准确率），并额外使用相同的预热和学习率进行 200 个 epoch 的微调（这为 Path-X 的 FlashAttention 增加了大约 4 个点的准确率，但模型随后开始过拟合）。

E.4 与 Apex FMHA 的比较

我们将我们的方法/实现与 Apex FMHA (https://github.com/NVIDIA/apex/tree/ master/apex/contrib/csrc/fmha) 进行比较。

当我们开始这个项目时，Apex FMHA 是当时（我们所知）最快的注意力实现，专为长度最多 512 的短序列量身定制。事实上，截至 MLPerf 1.1 [58]，几乎所有在 Nvidia GPU 上运行的 BERT 训练基准 MLPerf 提交都使用 FMHA 作为其模型代码。

Table 7: Runtime (ms) of FlashAttention compared to FMHA by sequence length, with masking and dropout, measured on an A100-SXM4-40GB GPU. Batch size 64, 16 heads, head dimension 64 (i.e., BERT-large size).

Attention Method	128	256	512
Apex FMHA forward	0.10	0.29	1.14
FlashAttention forward	**0.08**	**0.22**	**0.81**
Apex FMHA backward	**0.17**	**0.52**	**1.81**
FlashAttention backward	0.20	0.53	2.00
Apex FMHA forward + backward	**0.27**	0.81	2.95
FlashAttention forward + backward	0.28	**0.75**	**2.81**

FMHA targets BERT models, it only supports head dimension 64, and only runs on A100 GPUs. FMHA fuses the attention computation dropout(softmax(mask($\mathbf{QK}^\top$)))$\mathbf{V}$ into one CUDA kernel. In the forward pass, it stores the attention matrix softmax(mask($\mathbf{QK}^T$)) to HBM to be used in gradient computation. As a result, it does not offer substantial memory saving (though for shorter sequences memory footprint is often not a primary concern).

We use FMHA code as a starting point, and apply two well-established techniques (tiling and recomputation) to deal with long sequences and to save memory as mentioned in Section 3. As a result, we can support much longer sequences (e.g., up to length 64K). We also support more head dimensions (16, 32, 64, 128) and broader GPU types (all Turing and Ampere GPUs at the time of writing).

In Table 7, we compare the performance of FlashAttention and Apex FMHA for short sequences (as FMHA only supports sequence length at most 512). Generally FlashAttention is slightly faster than FMHA in the forward pass and slightly slower than FMHA in the backward pass. This is because we do not store the attention matrix in the forward pass and recompute it in the backward pass. Compared to FMHA, the overall runtime of FlashAttention is about 4% slower for sequence length 128, 8% faster for sequence length 256, and 5% faster for sequence length 512.

E.5 Speedup On Different Hardware and Configurations

Speedup varies between different types of GPU types and generations depending on HBM bandwidth and SRAM size. In this section, we profile FlashAttention speedup on different GPUs and configurations.

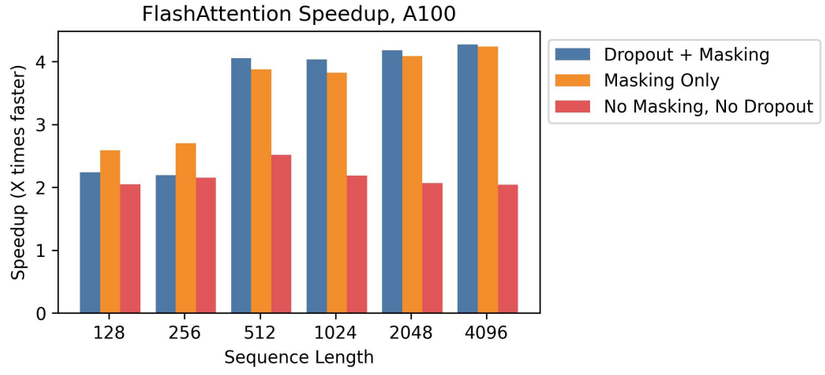


Figure 5: Speedup over standard PyTorch attention at different sequence lengths, on A100.

A100 Figure 5 shows speedup on an A100 GPU with batch size 8, head dimension 64, and 12 attention heads, across different sequence lengths. We generally see 2-4× speedup, and we see more speedup when using dropout and masking due to kernel fusion.

表 7：FlashAttention 与 FMHA 在不同序列长度下的运行时（毫秒），包含掩码和 Dropout，使用 A100-SXM4-40GB GPU 测量。批大小 64，16 个头，头维度 64（即 BERT-large 尺寸）。

注意力方法	128	256	512
Apex FMHA 正向	0.10	0.29	1.14
FlashAttention 正向	**0.08**	**0.22**	**0.81**
Apex FMHA 反向	**0.17**	**0.52**	**1.81**
FlashAttention 反向	0.20	0.53	2.00
Apex FMHA 正向 +反向	**0.27**	0.81	2.95
FlashAttention 正向 +反向	0.28	**0.75**	**2.81**

由于 FMHA 针对 BERT 模型，它仅支持头维度 64，并且仅在 A100 GPU 上运行。FMHA 将注意力计算 dropout(softmax(mask($\mathbf{QK}^\top$)))$\mathbf{V}$ 融合到一个 CUDA 内核中。在前向传递中，它将注意力矩阵 softmax(mask($\mathbf{QK}^T$)) 存储到 HBM 中以用于梯度计算。因此，它并未提供显著的内存节省（尽管对于较短的序列，内存占用通常不是主要问题）。

我们使用 FMHA 代码作为起点，并应用两种成熟的技巧（tiling 和重新计算）来处理长序列并节省内存，如第 3 节所述。结果，我们可以支持更长的序列（例如，长达 64K）。我们还支持更多的头维度（16、32、64、128）以及更广泛的 GPU 类型（截至编写时为所有 Turing 和 Ampere GPU）。

在表 7 中，我们比较了 FlashAttention 和 Apex FMHA 在短序列（因为 FMHA 仅支持最多 512 的序列长度）上的性能。通常情况下，FlashAttention 在正向传递中比 FMHA 稍快，在反向传递中比 FMHA 稍慢。这是因为我们在正向传递中不存储注意力矩阵，而是在反向传递中重新计算它。与 FMHA 相比，FlashAttention 的整体运行时在序列长度为 128 时约慢 4%，在序列长度为 256 时约快 8%，在序列长度为 512 时约快 5%。

E.5 在不同硬件和配置上的加速比

加速比因不同的GPU类型和代数而异，这取决于HBM带宽和SRAM大小。在本节中，我们分析了FlashAttention在不同GPU和配置上的加速比。

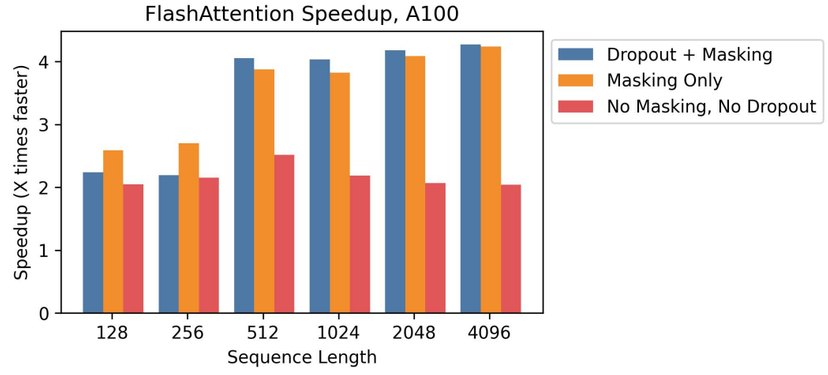


图5：在不同序列长度下，A100上FlashAttention相对于标准PyTorch注意力的加速比

A100 图5展示了在A100 GPU上，批大小为8、头维度为64、注意力头数为12的情况下，在不同序列长度上的加速比。我们通常看到2-4× 加速比，并且在使用Dropout和Masking时能看到更大的加速比，这是由于内核融合。

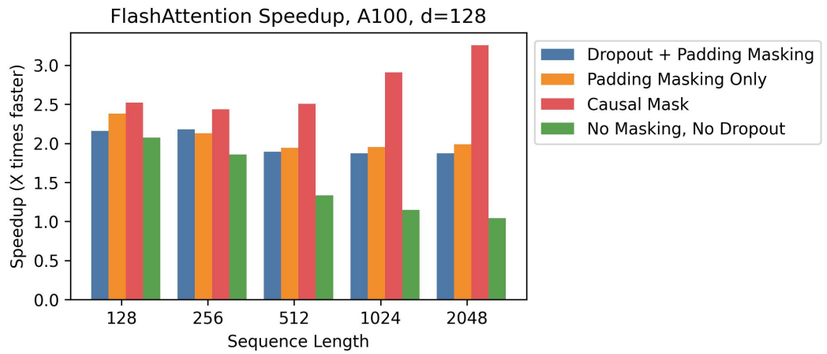


Figure 6: Speedup over standard PyTorch attention at different sequence lengths, on A100, with head dimension 128.

A100, Head Dimension 128 Speedup also changes when we increase the head dimension. Each block requires more memory, so we need to use smaller block sizes to fit into SRAM. Figure 6 shows speedup with head dimension 128 on an A100 (batch size 16, 12 heads). We see less speedup overall—but we can still see significant speedup (up to 3×) with a causal mask, where half the blocks are masked out.

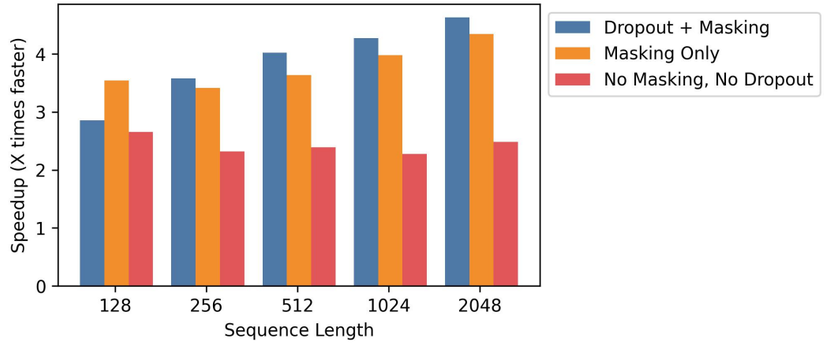


Figure 7: Speedup over standard PyTorch attention at different sequence lengths, on RTX 3090.

RTX 3090 Figure 7 shows speedup on an RTX 3090 GPU. Here, we use batch size 12 with 12 attention heads. We observe slightly higher speedups on the RTX 3090 (between 2.5-4.5×), since the memory bandwidth on an RTX 3090 is lower than on an A100 (roughly 900 GB/s vs. 1.5 TB/s).

T4 Figure 8 shows speedup on a T4 GPU. T4 SRAM is smaller than A100, so we need to make the block sizes smaller in FlashAttention. As a result, we observe less speedup on T4, which matches the IO complexity analysis in Section 3.2. T4 GPUs are commonly used for inference, so we also report speedup on the forward pass only.

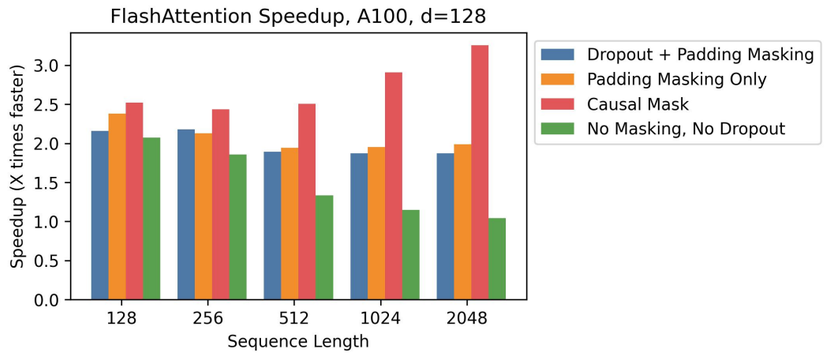


图6：在不同序列长度下，A100上相对于标准PyTorch注意力的加速比，头维度为128。

A100, 头维度 128 加速比在我们增加头维度时也会变化。每个块需要更多的内存，因此我们需要使用更小的块大小以适应 SRAM。图 6 显示了在 A100 上（批大小 16，12 个头）头维度为 128 时的加速比。我们观察到整体加速比较低——但我们仍然可以看到在使用因果掩码时存在显著加速比（高达 3×），其中一半的块被掩码掉。

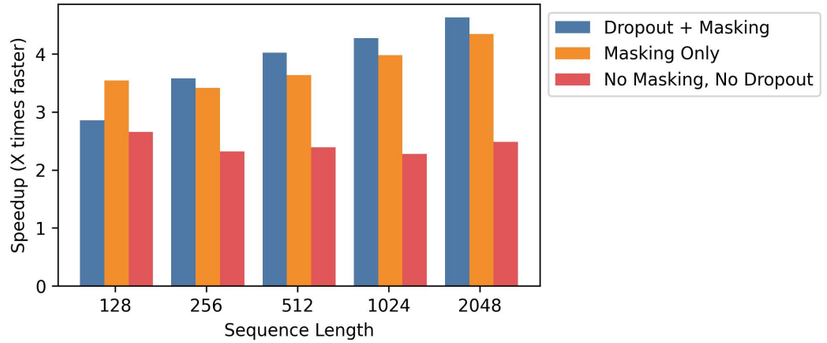
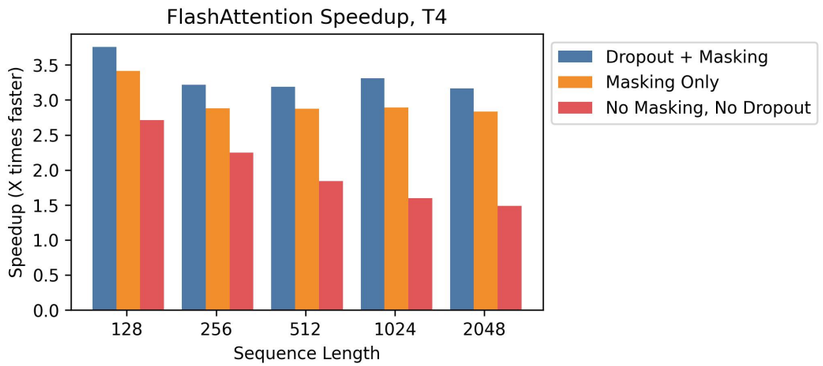


图7：在不同序列长度下，RTX 3090上相对于标准PyTorch注意力的加速比。

RTX 3090 图7显示了RTX 3090 GPU上的加速比。这里，我们使用12个批大小和12个注意力头。我们观察到RTX 3090上的加速比略高（在2.5-4.5×之间），因为RTX 3090的内存带宽低于A100（大约900 GB/s vs. 1.5 TB/s）。

T4 图8显示了在T4 GPU上的加速比。T4的SRAM比A100小，因此在FlashAttention中我们需要将块大小调小。结果，我们在T4上观察到的加速比较低，这与第3.2节的I/O复杂度分析相符。T4 GPU通常用于推理，所以我们仅报告了前向传递的加速比。



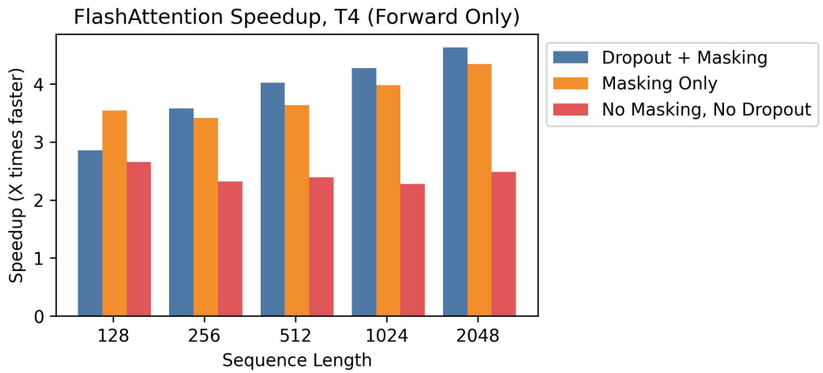


Figure 8: Speedup over standard PyTorch attention at different sequence lengths, on T4. **Top:** Combined forward pass + backward pass. **Bottom:** Forward pass only.

E.6 Full Benchmarking Results

We report the full benchmarking results and experimental details on A100.

Baselines We compare against reference implementations for exact attention from PyTorch/HuggingFace and Megatron, approximate attention, and sparse attention. For approximate attention, we compare against reference implementations of Reformer [51], Local Attention [68], Linformer Attention [84], Smyrf [19], and LongShortFormer (LSFormer) [94]. For sparse attention, we compare against reference implementations of Block-Sparse Attention form OpenAI [11], Longformer[3], and BigBird Attention [92]. For the approximate and sparse attention, we use a compression ratio of 1/8, or a compressed sequence length of 256, whichever is smaller.

Setup We measure runtime and memory usage of the attention computation with 8 heads of dimension 64, and batch size 16 on a machine with one A100 GPU with 40 GB of GPU HBM. We vary sequence length in our experiments. We compute attention on random vectors for **Q**, **K**, and **V** (we do not measure the projection from the hidden layer). For dropout, we use dropout 0.1; for masking, we use a padding mask with uniformly-random mask lengths between the total sequence length and the total sequence length minus 20. To measure runtime, we take the average of 100 measurements of the attention call. We only measure memory footprint once, since it does not vary between runs.

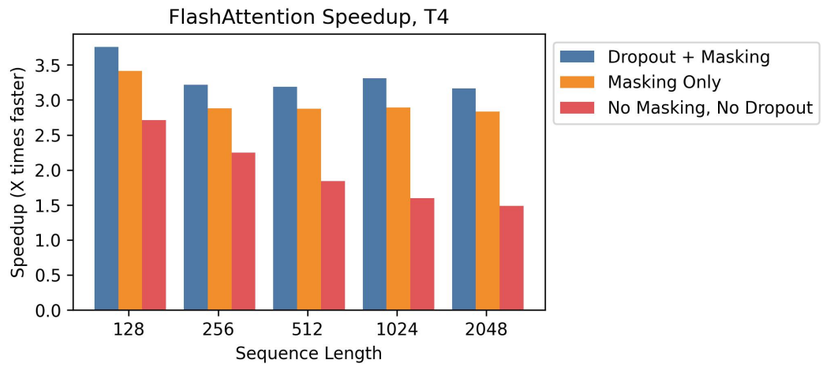


图8：在不同序列长度下，T4上相对于标准PyTorch注意力的加速比。**顶部：**组合前向传递 +后向传递。**底部：**仅前向传递。

E.6 完整基准测试结果

我们在A100上报告了完整的基准测试结果和实验细节。

基线我们与来自PyTorch/HuggingFace和Megatron的精确注意力参考实现、近似注意力和稀疏注意力进行了比较。对于近似注意力，我们与Reformer [51], 局部注意力 [68], Linformer注意力 [84], Smyrf [19], 和LongShortFormer (LSFormer) [94]的参考实现进行了比较。对于稀疏注意力，我们与来自OpenAI的Block-Sparse注意力 [11], Longformer[3], 和BigBird注意力 [92]的参考实现进行了比较。对于近似注意力和稀疏注意力，我们使用压缩率为1/8，或压缩后的序列长度为256，取较小值。

设置 我们在一台配备1块A100 GPU（含40 GB GPU HBM）的机器上，使用8个维度为64的注意力头和16的批大小，测量注意力计算的运行时和内存使用。我们实验中变化序列长度。我们对随机向量计算注意力，用于 **Q**、**K** 和**V** （我们不测量从隐藏层的投影）。对于dropout，我们使用dropout 0.1；对于掩码，我们使用填充掩码，其长度在总序列长度和总序列长度减去20之间均匀随机。为了测量运行时，我们对注意力调用进行100次测量并取平均值。我们只测量一次内存占用，因为它在不同运行之间没有变化。

Table 8: Pointers to results tables.

Dropout	Masking	Pass	Table
Yes	Yes	Forward	Table 9
Yes	Yes	Backward	Table 10
Yes	Yes	Combined	Table 11
No	Yes	Forward	Table 12
No	Yes	Backward	Table 13
No	Yes	Combined	Table 14
Yes	No	Forward	Table 15
Yes	No	Backward	Table 16
Yes	No	Combined	Table 17
No	No	Forward	Table 18
No	No	Backward	Table 19
No	No	Combined	Table 20
No	No	Memory Usage (Combined)	Table 21

Table 9: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
Local Attention	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer	0.67	0.52	0.69	0.71	1.65	3.18	6.15	12.16	24.17	52.39
Smyrf	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
Block Sparse	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FlashAttention	**0.04**	0.06	0.21	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
Block-Sparse FlashAttention	0.06	**0.06**	**0.06**	**0.12**	**0.44**	**0.86**	**1.70**	**3.29**	**6.55**	**13.34**

We report timing results on the forward pass, backward pass, and combined forward + backward pass. We measure each method with and without dropout, masking, or both—except for Block Sparse, Longformer, and BigBird. These methods did not successfully run the backward pass with masking due to a bug in external libraries, so we measured them without masking to be generous. We use FP16 for all measurements, except for Local Attention, whose implementation only supports FP32.

For each baseline, we increase sequence length until it runs out of memory on the GPU, except for the following exceptions: The Megatron implementation does not support sequence lengths longer than 2048. Block-Sparse (OpenAI) does not support sequence lengths longer than 4096. Longformer and BigBird do not support sequence lengths longer than 8092.

We measure memory usage on the combined forward + backward pass, without dropout or masking.

Results Table 8 summarizes all the experimental configurations and contains pointers to the results tables.

表 8：指向结果表的指针。

Dropout	Masking	Pass	Table
Yes	Yes	正向	表 9
Yes	Yes	反向	表 10
Yes	Yes	组合	表 11
No	Yes	正向	表12
No	Yes	反向	表13
No	Yes	组合	表14
Yes	No	正向	表15
Yes	No	反向	表16
Yes	No	组合	表17
No	No	正向	表18
No	No	反向	表19
No	No	组合	表20
No	No	内存使用（组合）	表21

表 9：不同序列长度下各种精确/近似/稀疏注意力机制的<code>前向传递</code>运行时间（毫秒），**包含Dropout和掩码**。最佳值<code>加粗</code>显示，次佳值<code>加下划线</code>显示。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
局部注意力	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer	0.67	0.52	0.69	0.71	1.65	3.18	6.15	12.16	24.17	52.39
Smyrf	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
块稀疏	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FlashAttention	**0.04**	0.06	0.21	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
块稀疏FlashAttention	0.06	**0.06**	**0.06**	**0.12**	**0.44**	**0.86**	**1.70**	**3.29**	**6.55**	**13.34**

我们报告了前向传递、后向传递和组合前向 + 后向传递的计时结果。我们测量每种方法有无dropout、掩码或两者的情况——除了块稀疏、Longformer和BigBird。由于外部库中的bug，这些方法未能成功运行带掩码的后向传递，因此我们为了宽厚起见，测量了它们不带掩码的情况。我们所有测量都使用FP16，除了局部注意力，其实现仅支持FP32。

对于每个基线，我们增加序列长度，直到它在GPU上耗尽内存，除了以下例外：Megatron实现不支持超过2048的序列长度。Block-Sparse（OpenAI）不支持超过4096的序列长度。Longformer和BigBird不支持超过8092的序列长度。

我们测量m　在组合的正向和反向传递上测量内存使用，不包含Dropout或掩码　g.

结果 表8总结了所有实验配置，并包含指向结果表的信息。

Table 10: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
Local Attention	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	36.53	-
Smyrf	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
Block Sparse	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FlashAttention	**0.30**	**0.26**	0.68	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
Block-Sparse FlashAttention	**0.30**	0.27	**0.29**	**0.59**	**1.50**	**2.94**	**5.82**	**11.85**	**23.98**	**47.61**

Table 11: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
Local Attention	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	61.47	-
Smyrf	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.43**	**0.41**	0.95	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
Block-Sparse FlashAttention	0.44	0.44	**0.45**	**0.89**	**1.95**	**4.12**	**7.64**	**16.60**	**32.73**	**64.11**

Table 12: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
Local Attention	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer	0.63	0.50	0.67	0.65	1.36	2.60	5.04	9.92	19.69	43.47
Smyrf	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.03**	**0.04**	0.17	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
Block-Sparse FlashAttention	0.05	**0.04**	**0.05**	**0.11**	**0.35**	**0.68**	**1.33**	**2.54**	**5.34**	**10.73**

Table 13: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
Local Attention	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	32.67	72.11
Smyrf	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.21**	**0.22**	0.62	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
Block-Sparse FlashAttention	0.22	0.22	**0.26**	**0.57**	**1.55**	**3.13**	**5.98**	**12.21**	**23.49**	**47.85**

表 10：不同序列长度下各种精确/近似/稀疏注意力机制的<code>反向传播运行时间</code>(毫秒)，**包含Dropout和掩码**。最佳值<code>加粗</code>显示，次佳值<code>加下划线</code>显示。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
局部注意力	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	36.53	-
Smyrf	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
块稀疏	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FlashAttention	**0.30**	**0.26**	0.68	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
块稀疏FlashAttention	**0.30**	0.27	**0.29**	**0.59**	**1.50**	**2.94**	**5.82**	**11.85**	**23.98**	**47.61**

表 11：不同序列长度下各种精确/近似/稀疏注意力机制的 + 前向传递和后向传递运行时间（毫秒），**带有Dropout和掩码**。最佳值加粗显示，次佳值加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
局部注意力	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	61.47	-
Smyrf	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
块稀疏	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.43**	**0.41**	0.95	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
块稀疏FlashAttention	0.44	0.44	**0.45**	**0.89**	**1.95**	**4.12**	**7.64**	**16.60**	**32.73**	**64.11**

表12：序列长度下各种精确/近似/稀疏注意力机制的前向传递运行时间（毫秒），**带掩码**。最佳以**粗体**显示，次佳加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
局部注意力	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer	0.63	0.50	0.67	0.65	1.36	2.60	5.04	9.92	19.69	43.47
Smyrf	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
块稀疏	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.03**	**0.04**	0.17	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
块稀疏FlashAttention	0.05	**0.04**	**0.05**	**0.11**	**0.35**	**0.68**	**1.33**	**2.54**	**5.34**	**10.73**

表13：不同序列长度下各种精确/近似/稀疏注意力机制的<code>反向传播运行时间</code>（毫秒），**带掩码**。最佳值<code>加粗</code>显示，次佳值<code>加下划线</code>显示。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
局部注意力	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	32.67	72.11
Smyrf	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
块稀疏	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.21**	**0.22**	0.62	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
块稀疏FlashAttention	0.22	0.22	**0.26**	**0.57**	**1.55**	**3.13**	**5.98**	**12.21**	**23.49**	**47.85**

Table 14: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
Local Attention	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	53.36	117.56
Smyrf	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.32**	**0.30**	0.83	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
Block-Sparse FlashAttention	0.34	0.34	**0.36**	**0.69**	**1.85**	**3.89**	**7.16**	**14.85**	**30.46**	**60.03**

Table 15: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
Local Attention	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer	0.47	0.37	0.49	**0.52**	1.37	2.65	5.12	10.13	20.25	44.16
Smyrf	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
Block Sparse	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FlashAttention	**0.10**	**0.10**	**0.22**	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
Block-Sparse FlashAttention	0.54	0.51	0.68	0.61	**0.67**	**1.10**	**1.89**	**3.71**	**7.18**	**14.41**

Table 16: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
Local Attention	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer	0.84	0.74	0.79	0.85	2.28	4.37	8.66	17.02	33.78	-
Smyrf	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
Block Sparse	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FlashAttention	**0.15**	0.18	0.58	1.86	6.50	26.21	104.27	416.10	1661.92	6643.01
Block-Sparse FlashAttention	0.17	**0.17**	**0.17**	**0.40**	**1.10**	**2.04**	**4.43**	**9.33**	**18.28**	**37.31**

Table 17: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.66	0.67	1.43	4.82	17.47	67.29	-	-	-	-
Megatron	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
Local Attention	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	55.00	-
Smyrf	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.35**	**0.36**	**0.80**	2.52	9.16	36.70	146.13	583.45	2332.01	9323.63
Block-Sparse FlashAttention	0.91	0.83	0.94	**0.92**	**1.83**	**3.50**	**7.02**	**13.56**	**26.71**	**53.92**

表14：不同序列长度下各种精确/近似/稀疏注意力机制的<code>前向传递</code>和<code>后向传递</code>运行时间（毫秒），**带掩码**。最佳值加粗显示，次佳值加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
局部注意力	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	53.36	117.56
Smyrf	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
块稀疏	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.32**	**0.30**	0.83	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
块稀疏FlashAttention	0.34	0.34	**0.36**	**0.69**	**1.85**	**3.89**	**7.16**	**14.85**	**30.46**	**60.03**

表15：不同序列长度下各种精确/近似/稀疏注意力机制的<code>前向传递</code>运行时间（毫秒），**含Dropout**。最佳值<code>加粗</code>显示，次佳值<code>下划线</code>显示。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
局部注意力	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer	0.47	0.37	0.49	**0.52**	1.37	2.65	5.12	10.13	20.25	44.16
Smyrf	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
块稀疏	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FlashAttention	**0.10**	**0.10**	**0.22**	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
块稀疏FlashAttention	0.54	0.51	0.68	0.61	**0.67**	**1.10**	**1.89**	**3.71**	**7.18**	**14.41**

表16：反向传播运行时间（毫秒）随序列长度变化的各种精确/近似/稀疏注意力机制，**含Dropout**。最佳值**加粗**显示，次佳值<u>下划线</u>显示。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
局部注意力	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer	0.84	0.74	0.79	0.85	2.28	4.37	8.66	17.02	33.78	-
Smyrf	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
块稀疏	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FlashAttention	**0.15**	0.18	0.58	1.86	6.50	26.21	104.27	416.10	1661.92	6643.01
块稀疏FlashAttention	0.17	**0.17**	**0.17**	**0.40**	**1.10**	**2.04**	**4.43**	**9.33**	**18.28**	**37.31**

表 17：不同序列长度下各种精确/近似/稀疏注意力机制的<code>前向传递</code>和<code>后向传递</code>运行时间（毫秒），**带有Dropout**。最佳值加粗显示，次佳值加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.66	0.67	1.43	4.82	17.47	67.29	-	-	-	-
Megatron	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
局部注意力	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	55.00	-
Smyrf	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
块稀疏	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.35**	**0.36**	**0.80**	2.52	9.16	36.70	146.13	583.45	2332.01	9323.63
块稀疏FlashAttention	0.91	0.83	0.94	**0.92**	**1.83**	**3.50**	**7.02**	**13.56**	**26.71**	**53.92**

Table 18: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.21	0.22	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron	0.24	0.26	0.42	1.33	4.28	-	-	-	-	-
Reformer	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
Local Attention	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer	0.46	0.36	0.45	**0.50**	1.09	2.09	4.01	7.90	15.70	35.40
Smyrf	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.08**	**0.09**	**0.18**	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
Block-Sparse FlashAttention	0.56	0.52	0.63	0.65	**0.61**	**0.96**	**1.69**	**3.02**	**5.69**	**11.77**

Table 19: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
Local Attention	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer	0.69	0.76	0.69	0.80	2.04	3.88	7.67	15.04	30.11	63.15
Smyrf	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.11**	0.16	0.52	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
Block-Sparse FlashAttention	0.11	**0.12**	**0.16**	**0.38**	**1.20**	**2.34**	**4.69**	**9.10**	**18.74**	**37.04**

Table 20: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron	0.74	0.65	1.23	3.80	13.21	-	-	-	-	-
Reformer	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
Local Attention	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer	1.31	1.25	1.30	1.29	3.20	6.10	11.93	23.39	46.72	100.52
Smyrf	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.31**	**0.31**	**0.73**	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
Block-Sparse FlashAttention	0.74	0.77	0.82	**0.88**	**1.71**	**3.21**	**6.56**	**12.60**	**24.93**	**50.39**

Table 21: Memory usage (MB) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	36	104	336	1184	4416	17024	-	-	-	-
Megatron	36	104	336	1184	4416	-	-	-	-	-
Reformer	377	754	1508	3016	6033	12067	24134	-	-	-
Local Attention	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer	25	52	114	287	832	1652	3292	6572	13132	26252
Smyrf	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer	72	152	333	796	2540	5068	10125	20240	-	-
Block Sparse	33	82	228	408	910	2401	-	-	-	-
Longformer	30	61	124	277	681	1370	2748	-	-	-
BigBird	33	66	131	294	708	1431	2872	-	-	-
FlashAttention	**22**	**44**	**104**	**209**	**418**	**836**	**1672**	**3344**	**6688**	**13376**
Block-Sparse FlashAttention	22	44	104	209	418	836	1672	3344	6690	13384

表18：不同序列长度下各种精确/近似/稀疏注意力机制的<code>前向传递</code>运行时间（毫秒）。<code>最佳</code>以<code>**粗体**</code>显示，<code>次佳</code>加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.21	0.22	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron	0.24	0.26	0.42	1.33	4.28	-	-	-	-	-
Reformer	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
局部注意力	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer	0.46	0.36	0.45	**0.50**	1.09	2.09	4.01	7.90	15.70	35.40
Smyrf	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
块稀疏	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.08**	**0.09**	**0.18**	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
块稀疏FlashAttention	0.56	0.52	0.63	0.65	**0.61**	**0.96**	**1.69**	**3.02**	**5.69**	**11.77**

表19：不同序列长度下各种精确/近似/稀疏注意力机制的反向传播运行时间（毫秒）。最佳加粗显示，次佳加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
局部注意力	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer	0.69	0.76	0.69	0.80	2.04	3.88	7.67	15.04	30.11	63.15
Smyrf	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
块稀疏	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.11**	0.16	0.52	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
块稀疏FlashAttention	0.11	**0.12**	**0.16**	**0.38**	**1.20**	**2.34**	**4.69**	**9.10**	**18.74**	**37.04**

表20：不同序列长度下各种精确/近似/稀疏注意力机制的前向传递和后向传递运行时间（毫秒）。最佳值加粗显示，次佳值加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron	0.74	0.65	1.23	3.80	13.21	-	-	-	-	-
Reformer	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
局部注意力	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer	1.31	1.25	1.30	1.29	3.20	6.10	11.93	23.39	46.72	100.52
Smyrf	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
块稀疏	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.31**	**0.31**	**0.73**	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
块稀疏FlashAttention	0.74	0.77	0.82	**0.88**	**1.71**	**3.21**	**6.56**	**12.60**	**24.93**	**50.39**

表21：不同序列长度下各种精确/近似/稀疏注意力机制的内存使用（MB）。**加粗**为最佳，次佳加下划线。

注意力方法	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch注意力	36	104	336	1184	4416	17024	-	-	-	-
Megatron	36	104	336	1184	4416	-	-	-	-	-
Reformer	377	754	1508	3016	6033	12067	24134	-	-	-
局部注意力	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer	25	52	114	287	832	1652	3292	6572	13132	26252
Smyrf	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer	72	152	333	796	2540	5068	10125	20240	-	-
块稀疏	33	82	228	408	910	2401	-	-	-	-
Longformer	30	61	124	277	681	1370	2748	-	-	-
BigBird	33	66	131	294	708	1431	2872	-	-	-
FlashAttention	**22**	**44**	**104**	**209**	**418**	**836**	**1672**	**3344**	**6688**	**13376**
块稀疏FlashAttention	22	44	104	209	418	836	1672	3344	6690	13384